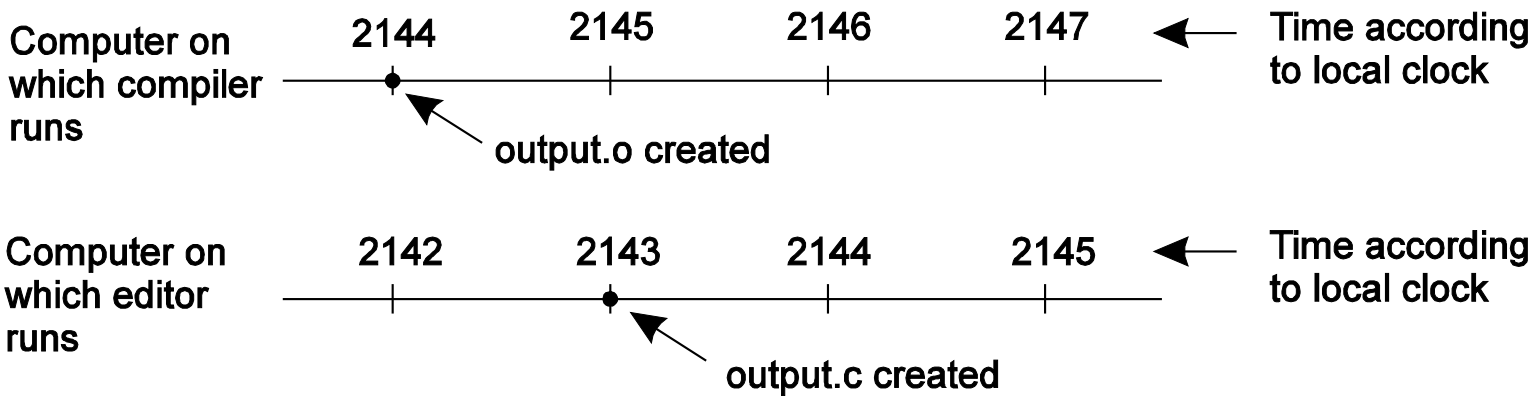
Fundamentals – Distributed Algorithms

Time and Global States

Introduction

- Recall one fundamental characteristic of distributed systems: loosely coupled
  - No globally shared clock – computers have their own physical clocks that deviate from one another
- Many distributed applications depend on timing
  - UNIX **`make`** facility compiling source files



  - Electronic commerce, e.g., auction and bidding

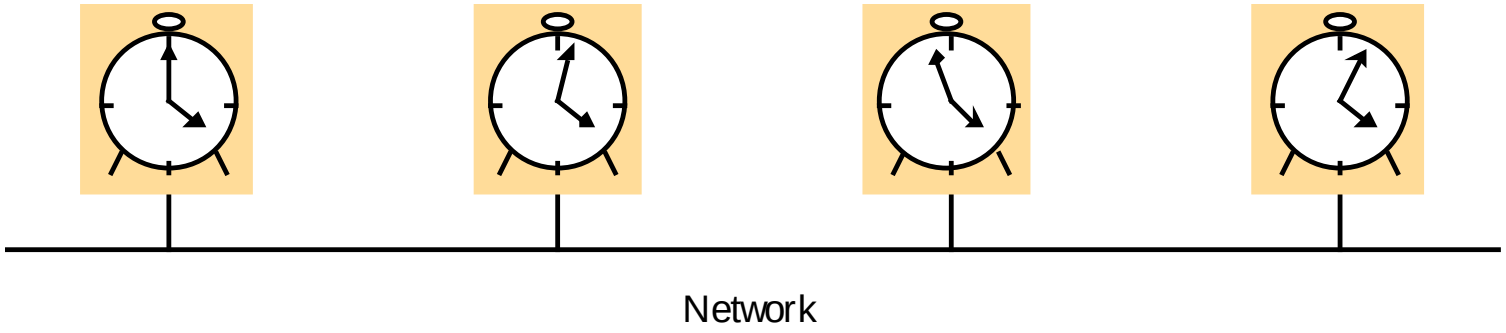
Introduction

- Solutions
  - Synchronize physical clocks of all computers
  - Design algorithms that do not rely on physical times

Outline

- Synchronizing Physical Clocks
- Causal Ordering and Logical Clocks
- Global States
- Distributed Debugging

Computer Clocks

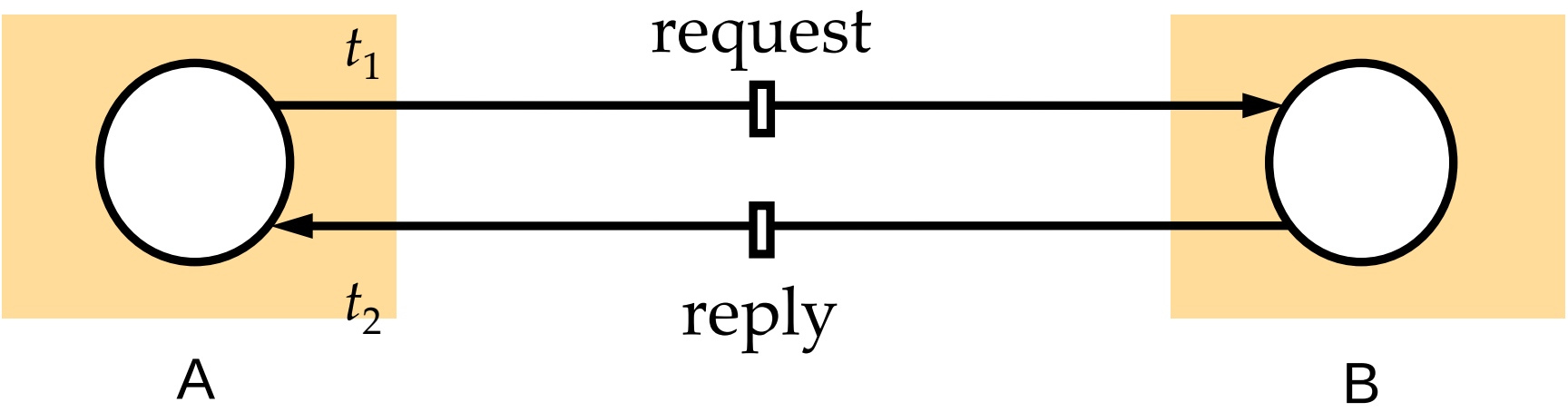


- Computers have their own physical clocks that deviate from one another
- Offset: instantaneous difference between the readings of two clocks
- Commonly used reference clock – Universal Coordinated Time (UTC)
  - UTC signals are broadcast regularly from land-based radio stations and satellites around the world

Computer Clocks

- Clock drift: clocks count time at different rates
- Drift rate: the change in offset between a clock and a perfect reference clock per unit of time measured by the reference clock
  - If a clock ticks 1,000,001 seconds while the reference clock ticks 1,000,000 seconds, drift rate = 1 μs/second
  - If a clock ticks 999,999 seconds while the reference clock ticks 1,000,000 seconds, drift rate = 1 μs/second
  - Drift rate of quartz crystal: about 10^{-6}

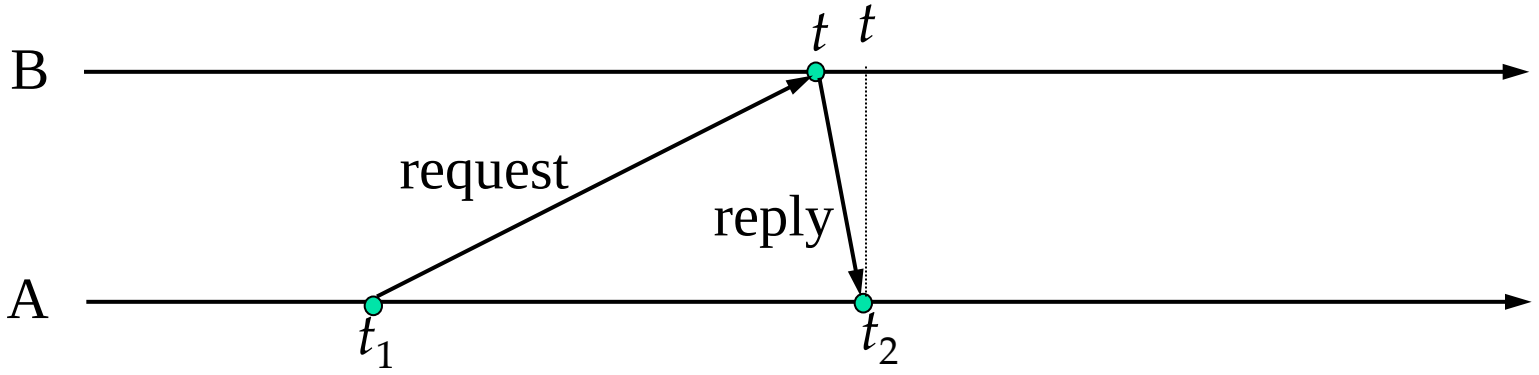
Cristian's Method



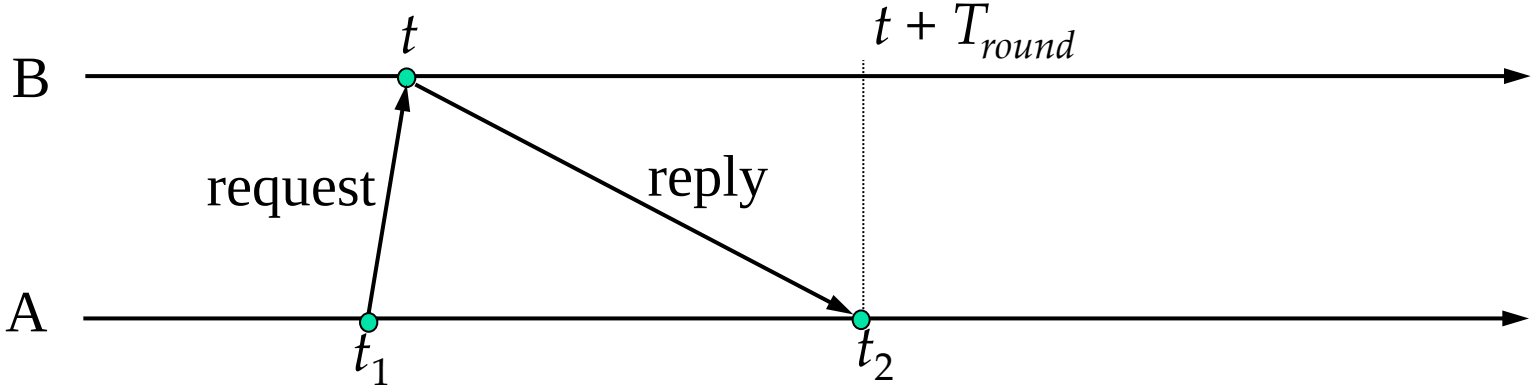
- Cristian's method
  - A requests the time of B
  - B replies with the time value t
  - A records the total round-trip time $T_{round} = t_2 - t_1$
  - A sets its clock to $t + T_{round}/2$
- So, what is the accuracy?

Cristian's Method

Case 1: transmission time of reply message approaches 0 → when A receives the reply, B's clock reading is ***t***



Case 2: transmission time of reply message approaches T_{round} → when A receives the reply, B's clock reading is ***t + T_{round}***



Cristian's Method

- When A receives m_t, the time reading on B should be in $[t, t + T_{round}]$
- A sets its clock to $t + T_{round}/2$

  → it is at most $T_{round}/2$ faster and is at most $T_{round}/2$ slower than B
- So, A is accurate within bound $T_{round}/2$
- To handle variability, make several requests to B and take the minimum value of T_{round} to give the most accurate estimate

Berkeley Algorithm

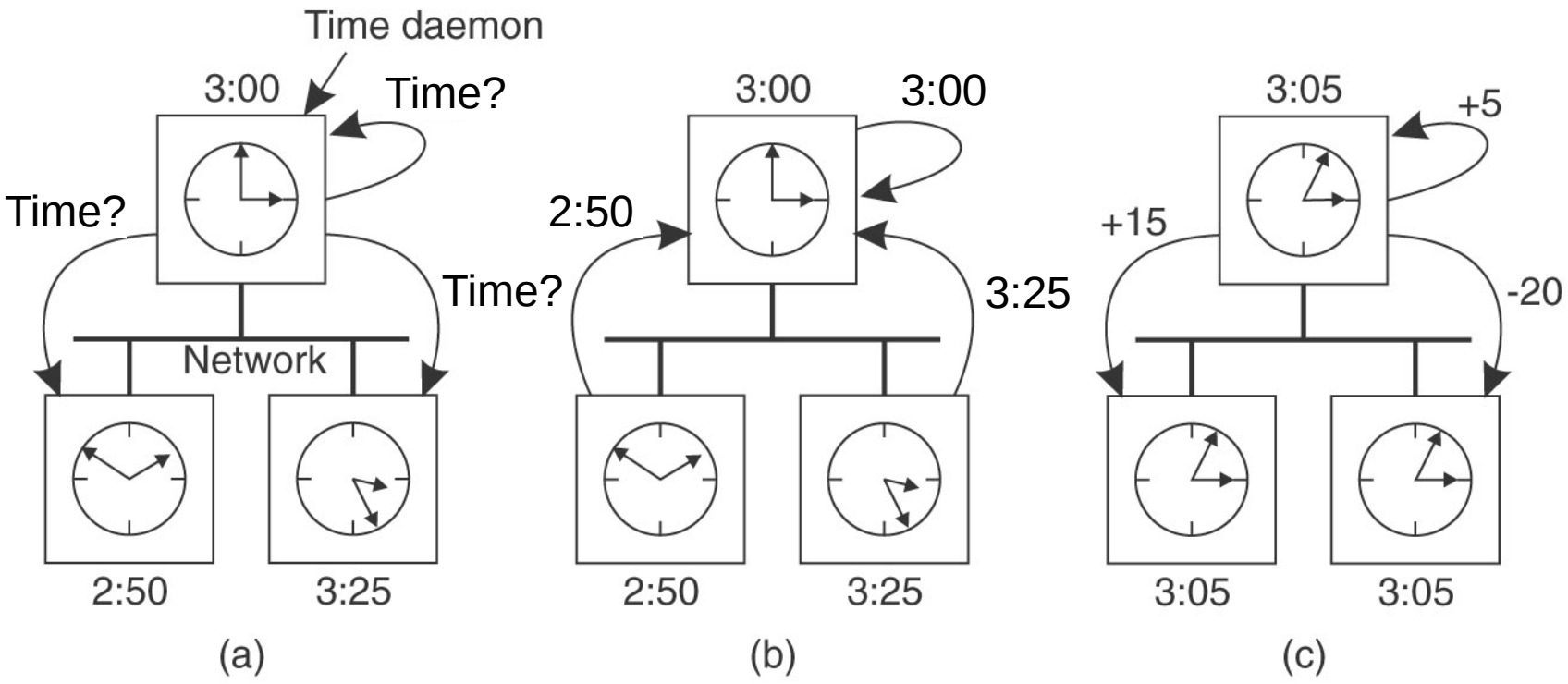
- Proposed by Gusella & Zetti for synchronizing a group of computers running Berkeley UNIX
- Master: a coordinator computer
- Slaves: other computers whose clocks are to be synchronized

Berkeley Algorithm

- How does it work?
  - Master periodically polls slaves
  - Slaves send their clock readings back
  - Master evaluates the local times of slaves by observing roundtrip times (similar to Cristian's method)
  - Master averages the values obtained (including its own clock reading)
  - Master sends the amount for adjustment to each slave whose clock requires adjustment

Berkeley Algorithm

- A simplified example



Berkeley Algorithm

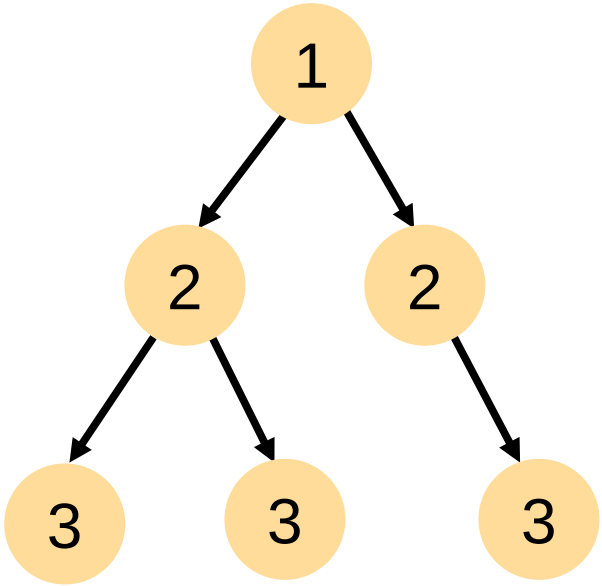
- Why average?
  - Average clock reading cancels out individual clocks' tendencies to run fast or slow
- Why sending amounts of adjustment to slaves?
  - Sending average clock reading back to slaves introduce further uncertainty due to message transmission time
- If master fails, another computer can be elected to take over its responsibility
- Empirical results: 15 computers on LAN (roundtrip time ~ 10 ms; clock drifts ~ 20 μs/s) synchronized to agree within 20-25 ms

Network Time Protocol

- Cristian's method and Berkeley algorithm
  - Primarily for intranets
  - Centralized design (not scalable)
- Network Time Protocol (NTP)
  - For distributing time information over the Internet

Network Time Protocol

- A network of servers structured hierarchically into a synchronization subnet
  - Primary servers (stratum 1) are connected directly to a time source (e.g., radio clock receiving UTC)
  - Stratum 2 servers are synchronized with primary servers
  - Stratum 3 servers are synchronized with stratum 2 servers
  - ……
  - Lowest-level servers execute in users' workstations



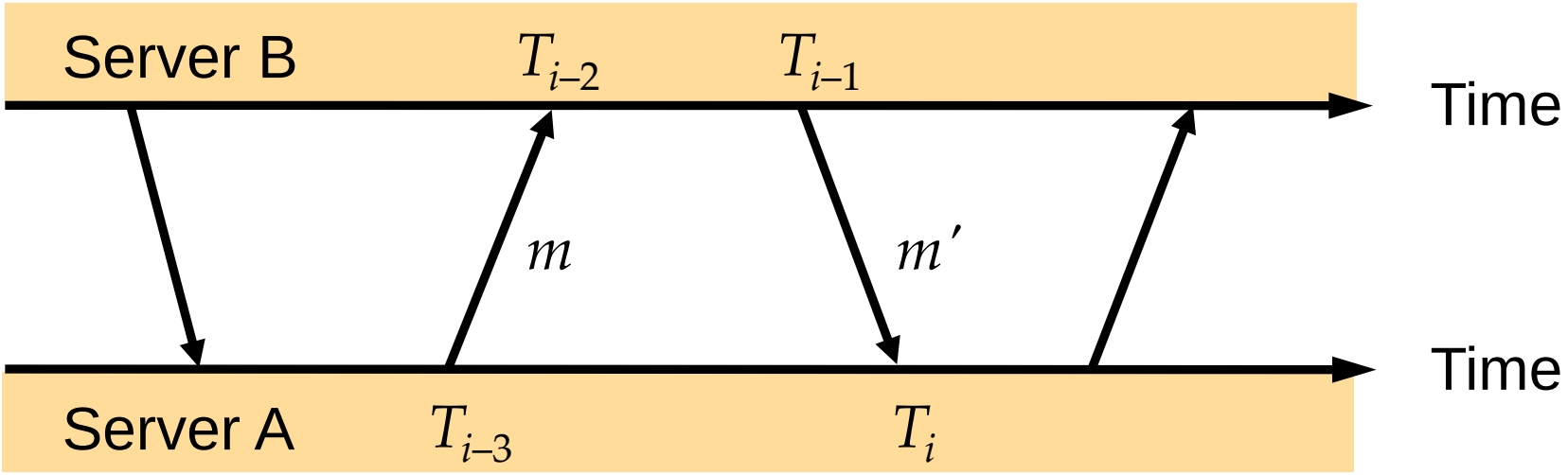
Arrows denote synchronization control, numbers denote strata

Network Time Protocol

- Fault-tolerance
  - Servers can reconfigure themselves if someone becomes unreachable
  - e.g., if a primary server's UTC source fails, it can become a stratum 2 server
  - e.g., if a stratum 2 server's normal source of synchronization (which is a primary server) fails or becomes unreachable, it may synchronize with another primary server
  - …

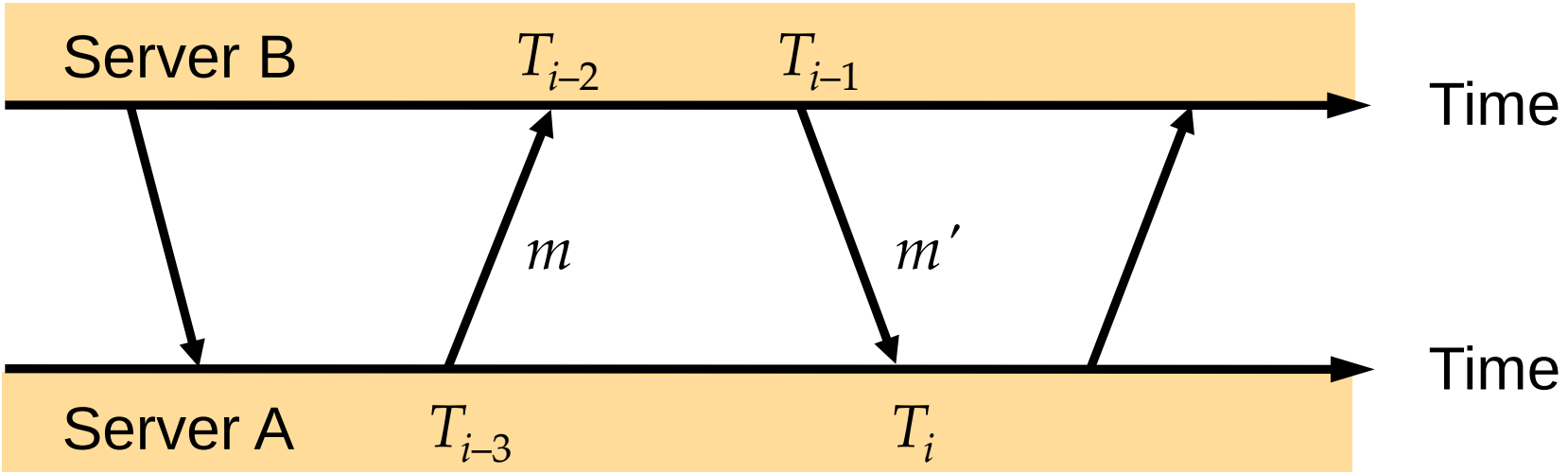
Network Time Protocol

- How to perform synchronization?
  - A pair of servers exchange messages bearing timing information

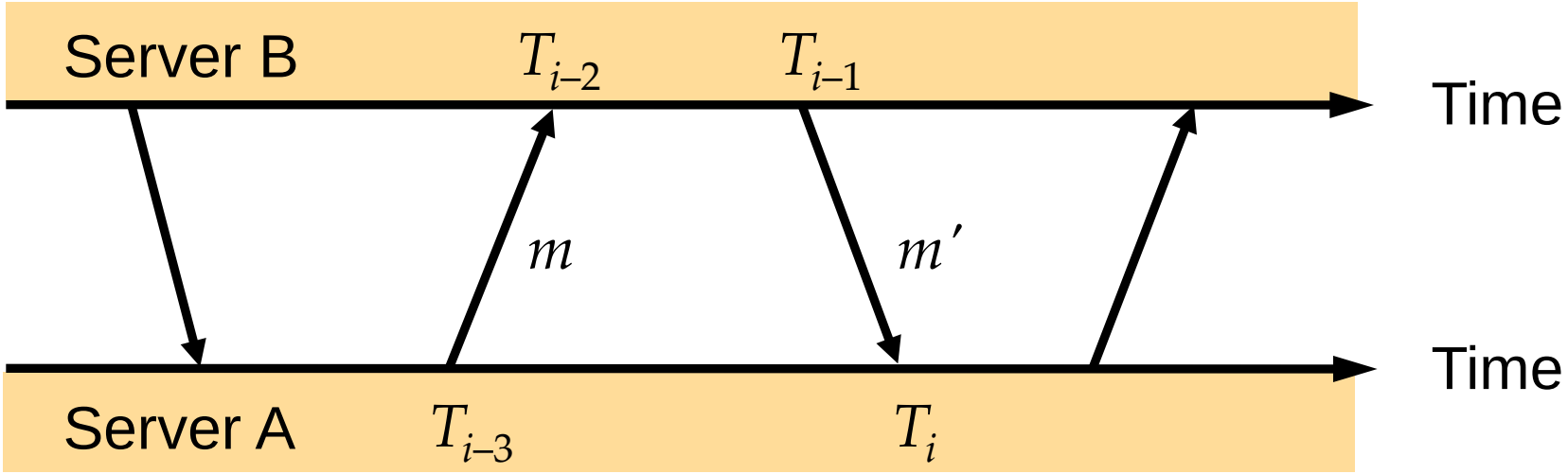


NTP Message Exchange

- Each NTP message contains:
  - Local times when the previous NTP message between the pair was sent and received
  - Local time when the current NTP message was sent
- Recipient of NTP message records local time of receipt

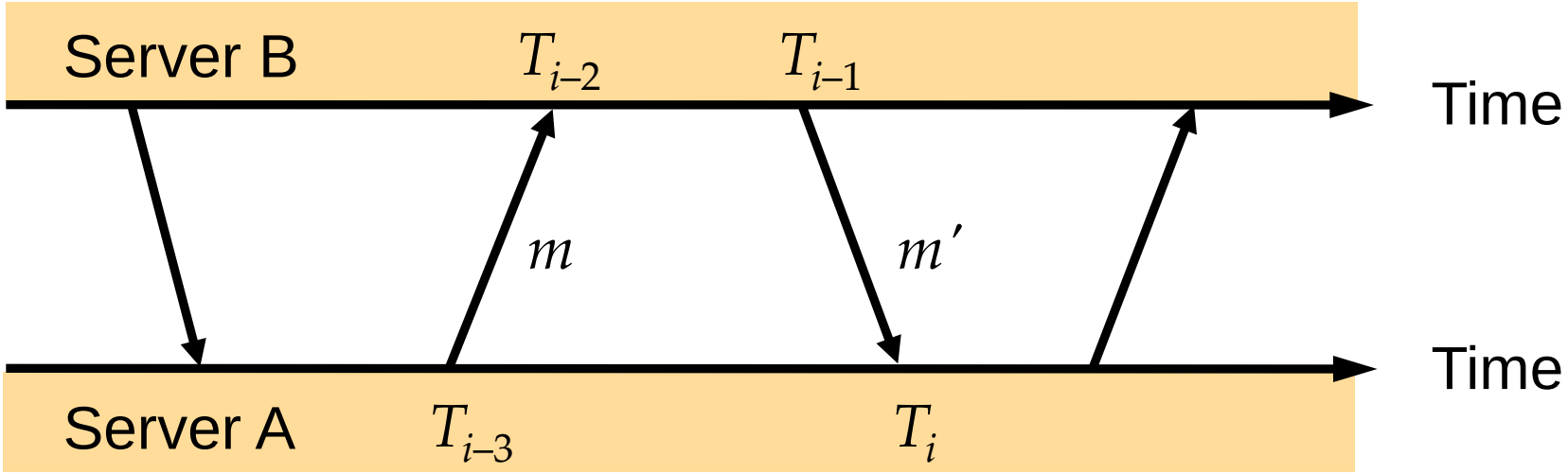


NTP Message Exchange



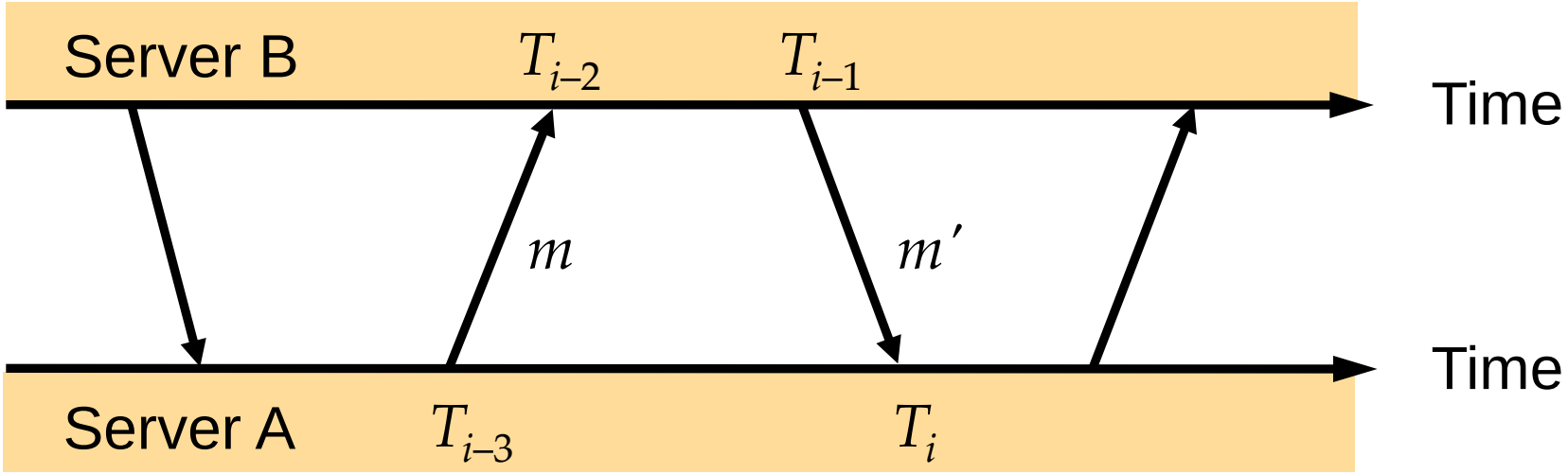
- For each pair of messages m and m' between 2 servers, how does A synchronize with B?
- Unlike Cristian's method, there can be a non-negligible delay between the arrival of m and the dispatch of m' at server B → consider this delay

NTP Message Exchange



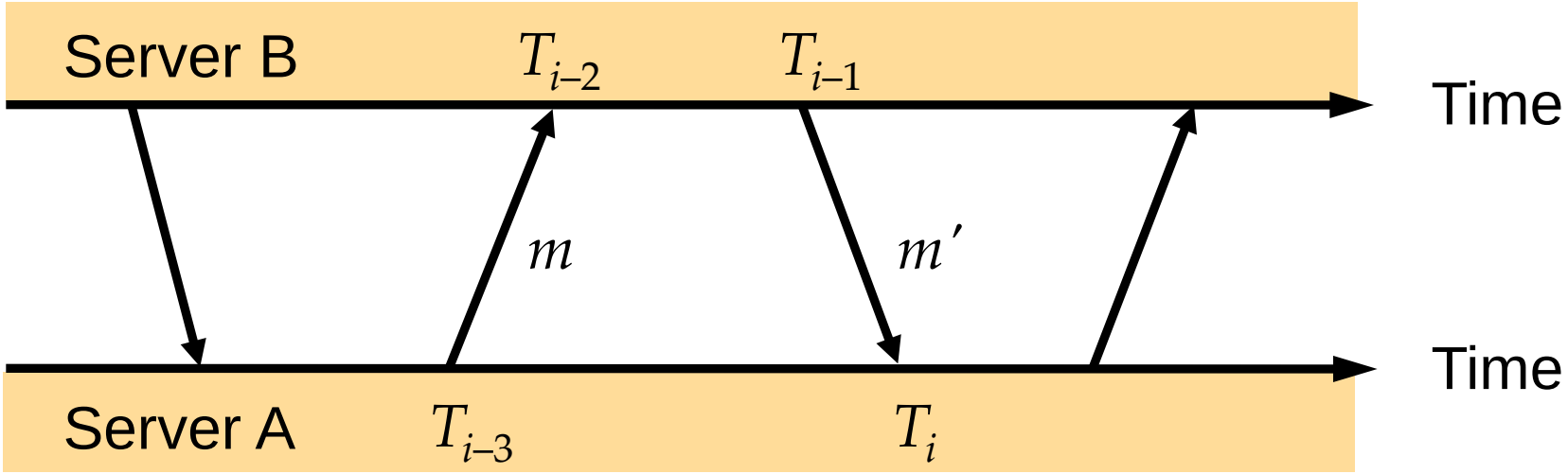
- Time between sending of m and receipt of m' is $T_i - T_{i-3}$
- Time between receipt of m and sending of m' is $T_{i-1} - T_{i-2}$
- So, total transmission time of m and m' is $(T_i - T_{i-3}) - (T_{i-1} - T_{i-2})$
- Thus, transmission time of m' is between 0 and $(T_i - T_{i-3}) - (T_{i-1} - T_{i-2})$

NTP Message Exchange



- If transmission time of m' approaches 0, when A receives m', B's clock reading is T_{i-1}
- If transmission time of m' approaches $(T_i - T_{i-3}) - (T_{i-1} - T_{i-2})$, when A receives m', B's clock reading is $T_{i-1} + (T_i - T_{i-3}) - (T_{i-1} - T_{i-2}) = T_i - T_{i-3} + T_{i-2}$

NTP Message Exchange



- If A wants to synchronize with B as accurately as possible, A should set its clock to ½ (T_{i-1} + ($T_i - T_{i-3} + T_{i-2}$)) = ½ ($T_i + T_{i-1} + T_{i-2} - T_{i-3}$) when receiving m'
- This setting has an accuracy of ± ½ ($T_{i-1} - (T_i - T_{i-3} + T_{i-2})$)

Network Time Protocol

- NTP applies data filtering to improve the accuracy of synchronization over time
  - Retain estimates of 8 most recent message exchanges and their accuracies
  - Choose the one with the highest accuracy for synchronization
- To choose a peer for primary use of synchronization, an NTP server engages in message exchanges with several of its peers and applies a peer-selection algorithm
- Empirical results: synchronization accuracy, 1 ms on LAN, tens of milliseconds over Internet

Revisit

- Solutions
  - Synchronize physical clocks of all computers
  - Design algorithms that do not rely on physical times
    - In many cases, knowing the absolute time is not necessary
    - What counts is the order in which related events at different processes occur

Outline

- Synchronizing Physical Clocks
- Causal Ordering and Logical Clocks
- Global States
- Distributed Debugging

System Model

- Distributed system
  - A collection of N processes $p_1, p_2, \ldots, p_N$
  - Each process executes on a single processor
  - Processors do not share memory
  - So, processes cannot communicate with one another except sending messages through the network
  - Communication channels are reliable
  - Asynchronous system: no bounds on process execution speeds, message transmission delays and clock drift rates

System Model

- State of a process
  - Values of all variables within the process
  - Values of any objects in the local OS environment that the process affects (e.g., files)
  - We refer to p_i's state as s_i
- Execution of a process
  - Process takes a series of actions, each of which is either sending a message, receiving a message, or an internal action that transforms the state of the process
  - An event: the occurrence of a single action that a process carries out as it executes

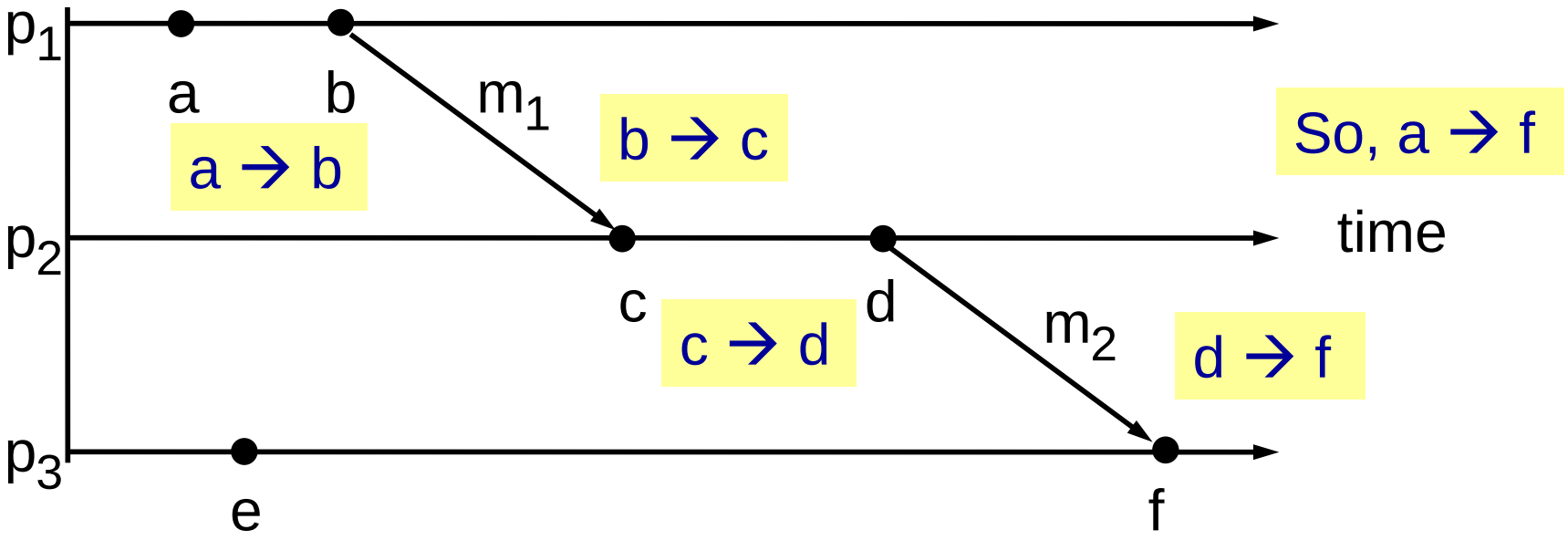
Causal Ordering

- In an asynchronous system where no globally shared clock exists, events can be ordered based on "cause-and-effect" (similar to physical causality)
  - Events within a single process p_i can be placed in a unique total ordering
    - Since each process executes on a single processor whose clock always advances, the sequence of events within a single process p_i is well defined
  - Whenever a message is sent between processes, the event of sending the message occurs before the event of receiving the message

Causal Ordering

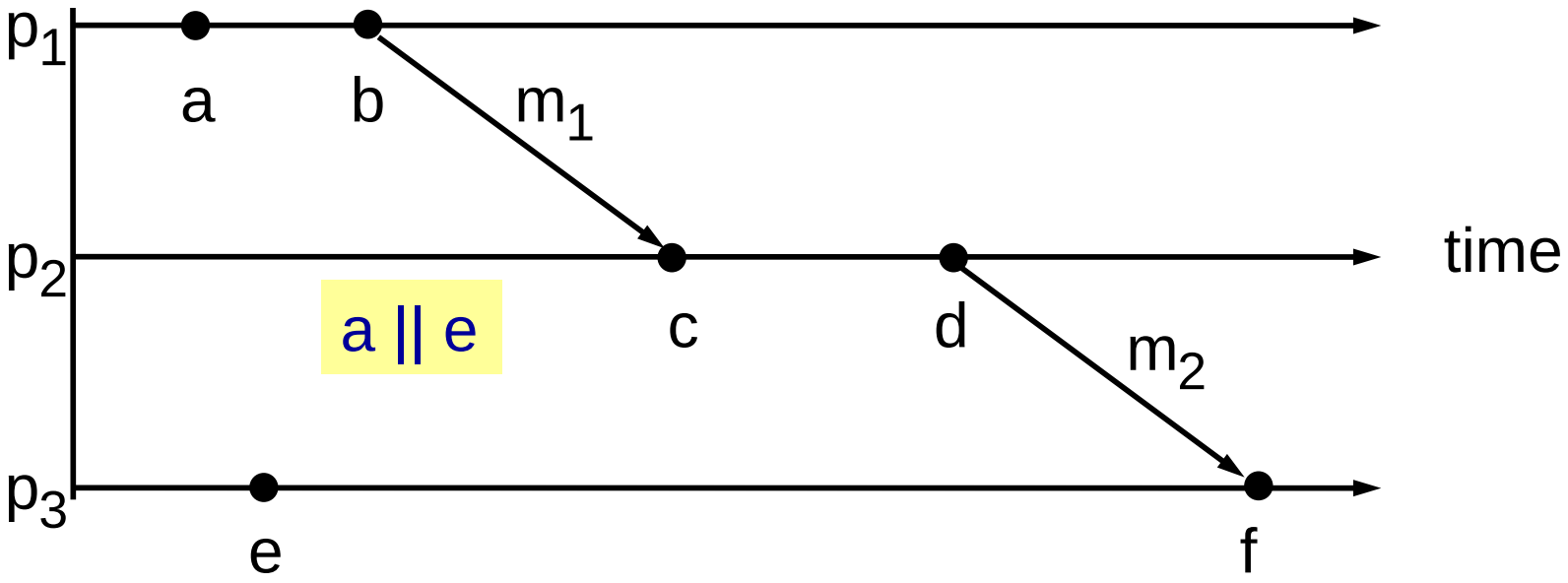
- Causal ordering (happened-before relation)
  - Denoted by "→"
    - $a \rightarrow b$ is read as "*a* happened-before *b*"
  - If e and e' are two events in the same process p_i and e occurs before e' at p_i , then $e \rightarrow e'$
  - For any message m, let $send(m)$ be the event of sending m, and $receive(m)$ be the event of receiving m, then $send(m) \rightarrow receive(m)$
    - Note that $send(m)$ and $receive(m)$ occur at different processes
  - If e, e', e'' are events such that $e \rightarrow e'$ and $e' \rightarrow e''$, then $e \rightarrow e''$

Example of Causal Ordering



- If $e \rightarrow e'$, we can find a sequence of events e_1, e_2, ..., e_n occurring at one or more processes such that
  - $e = e_1$, $e' = e_n$
  - For each $1 \leq i \leq n-1$, either e_i and e_{i+1} occur in succession at the same process, or there is a message m such that $e_i = send(m)$ and $e_{i+1} = receive(m)$

Example of Causal Ordering



- Causal ordering is a partial ordering – it may not be true that all pairs of distinct events are ordered
- If neither $e \rightarrow e'$ nor $e' \rightarrow e$, we say that they are concurrent and denote it by $e \parallel e'$

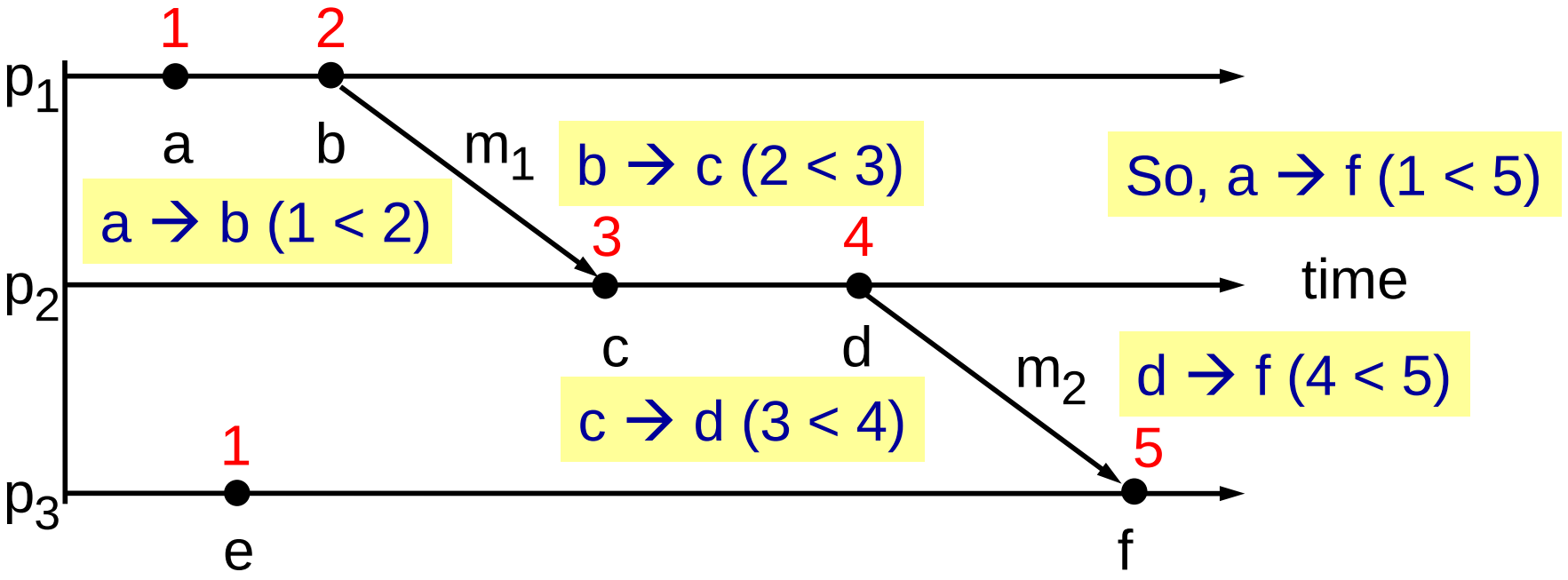
Lamport's Logical Clocks

- Logical clock
  - A monotonically increasing software counter whose value bears no particular relationship with real time
- Each process p_i keeps its own logical clock L_i to timestamp events

Lamport's Logical Clocks

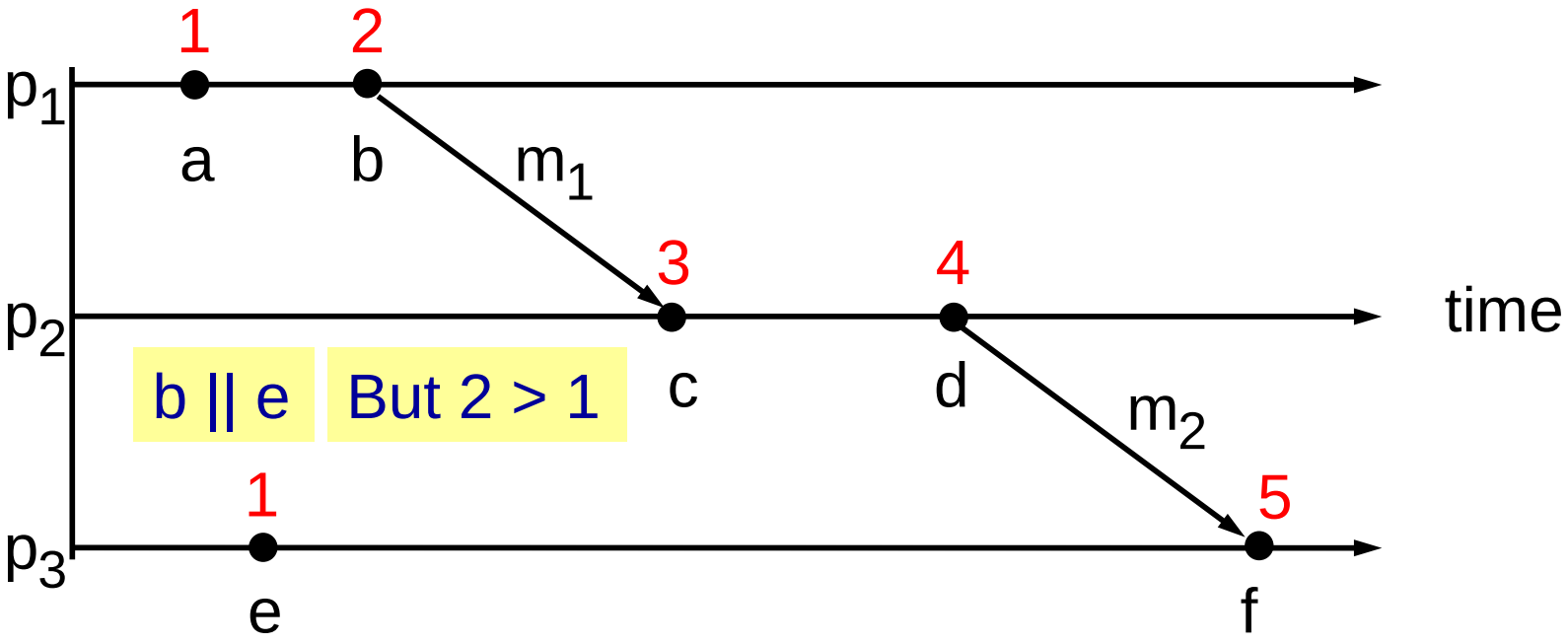
- How does it work? – update rules
  - All processes initialize their logical clocks to 0
  - Process p_i sets $L_i = L_i + 1$ before timestamping each event at p_i
  - When process p_i sends a message m, it piggybacks on m the value $t = L_i$
  - When another process p_j receives message (m, t), it sets $L_j = \max(L_j, t)$ before incrementing L_j to timestamp event *receive*(m)

Example of Lamport's Logical Clocks



- Denote the timestamp of event e by $L(e)$
- If $e \rightarrow e'$, then $L(e) < L(e')$ (How to prove it?)

Example of Lamport's Logical Clocks



- However, if $L(e) < L(e')$, we cannot infer that $e \rightarrow e'$

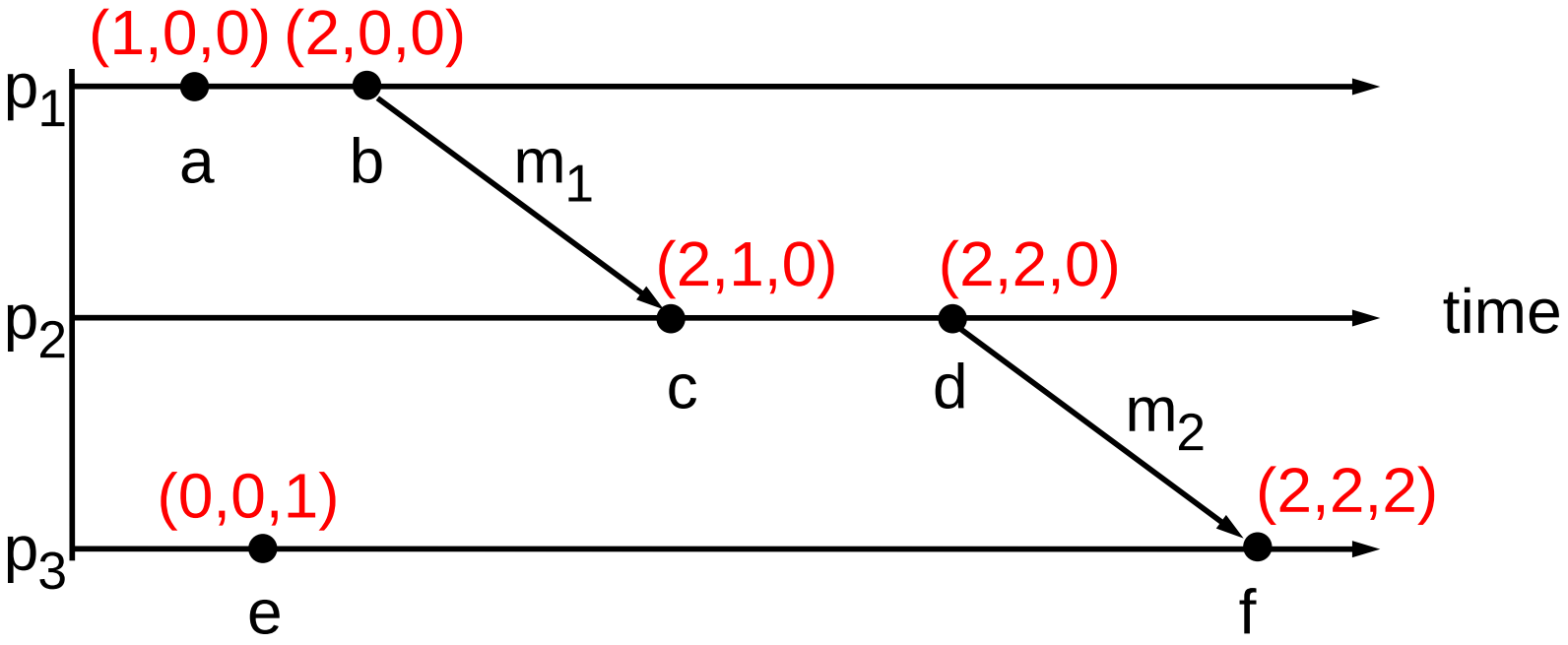
Vector Clocks

- Vector clock
  - A vector clock for a system of N processes is an array of N integers
- Each process p_i keeps its own vector clock V_i to timestamp events
  - Denote the kth component of clock V_i by $V_i[k]$

Vector Clocks

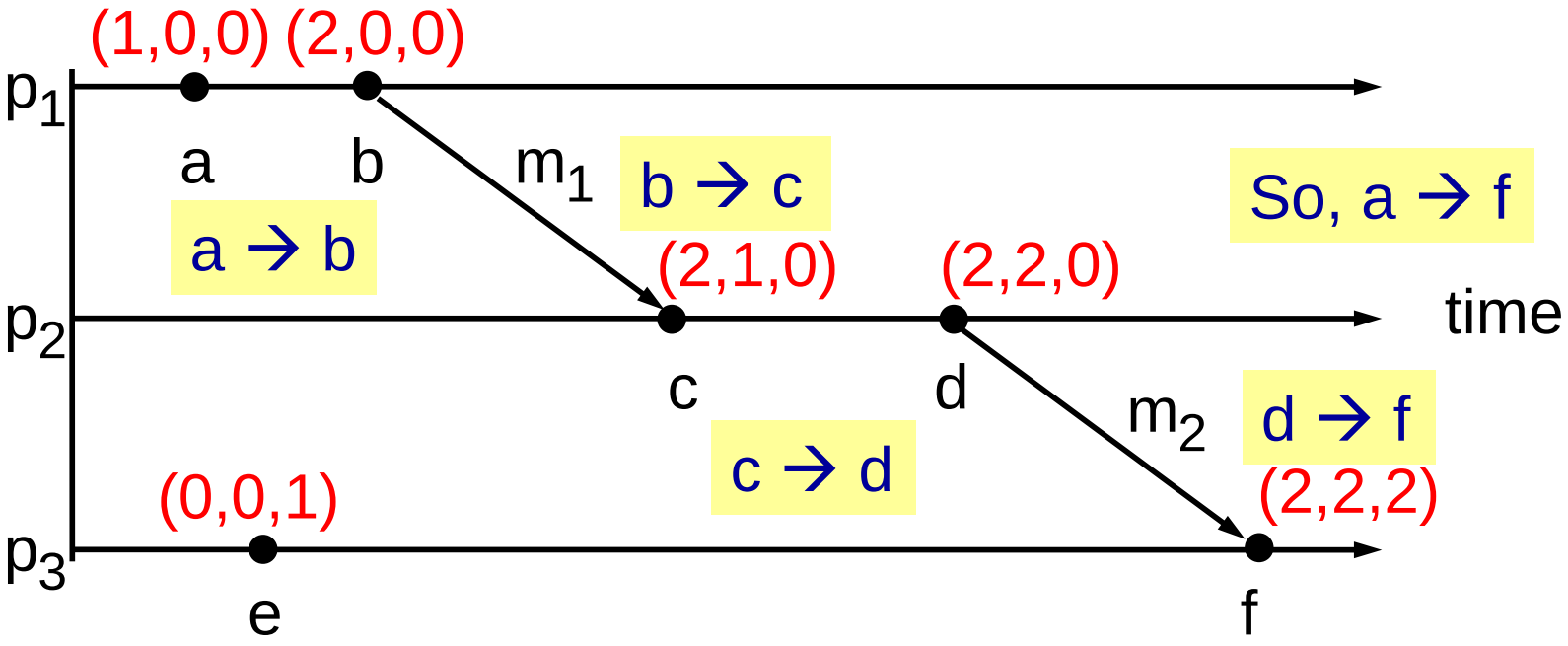
- How does it work? – update rules
  - All processes initialize their vector clocks to (0, 0, …, 0), i.e., $V_i[k] = 0$ for $i, k = 1, 2, \ldots, N$
  - Process p_i sets $V_i[i] = V_i[i] + 1$ before timestamping each event at p_i
  - When process p_i sends a message m, it piggybacks on m the value $t = V_i$
  - When another process p_j receives message (m, t), it sets $V_j[k] = \max(V_j[k], t[k])$ for $k = 1, 2, \ldots, N$ before incrementing $V_j[j]$ to timestamp event *receive*(m)

Example of Vector Clocks



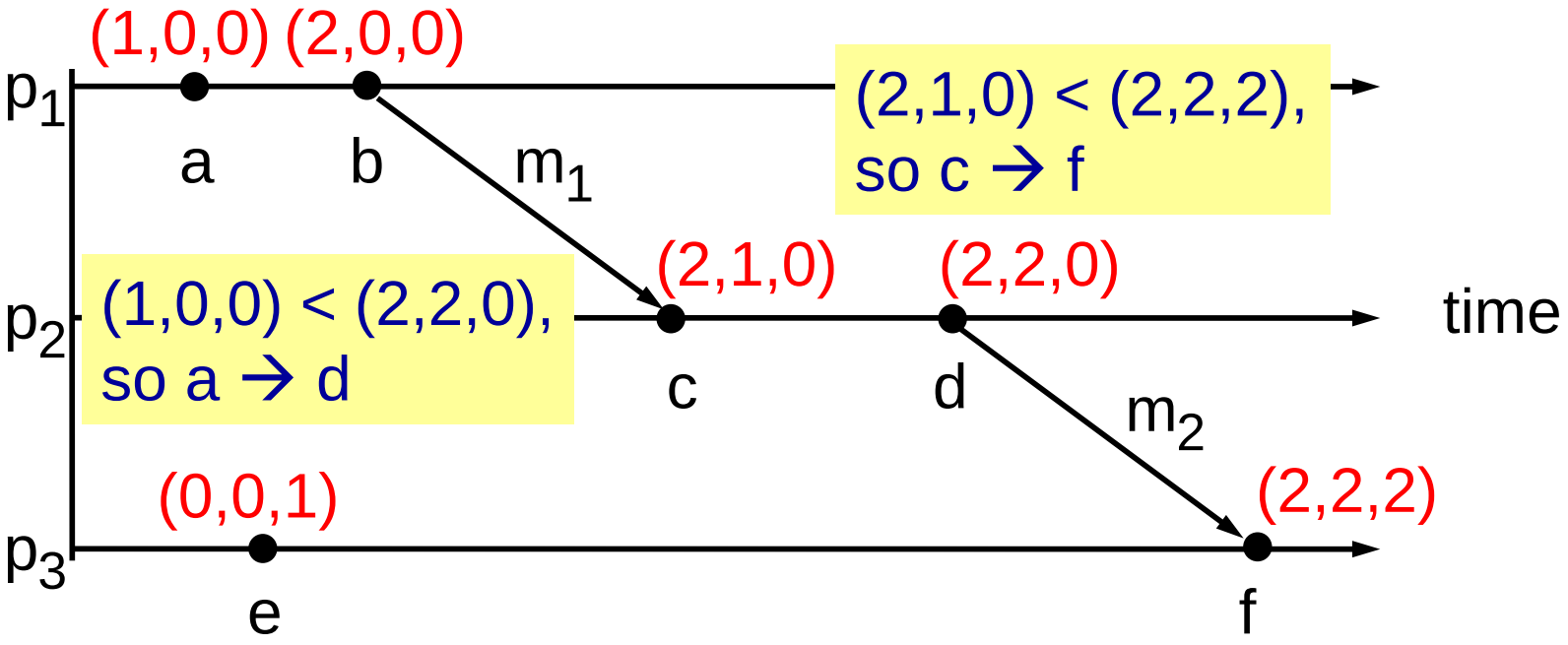
- Rules to compare vector timestamps V and V'
  - $V = V'$ if and only if $V[k] = V'[k]$ for $k = 1, 2, \ldots, N$
  - $V \leq V'$ if and only if $V[k] \leq V'[k]$ for $k = 1, 2, \ldots, N$
  - $V < V'$ if and only if $V \leq V'$ and $V \neq V'$

Example of Vector Clocks



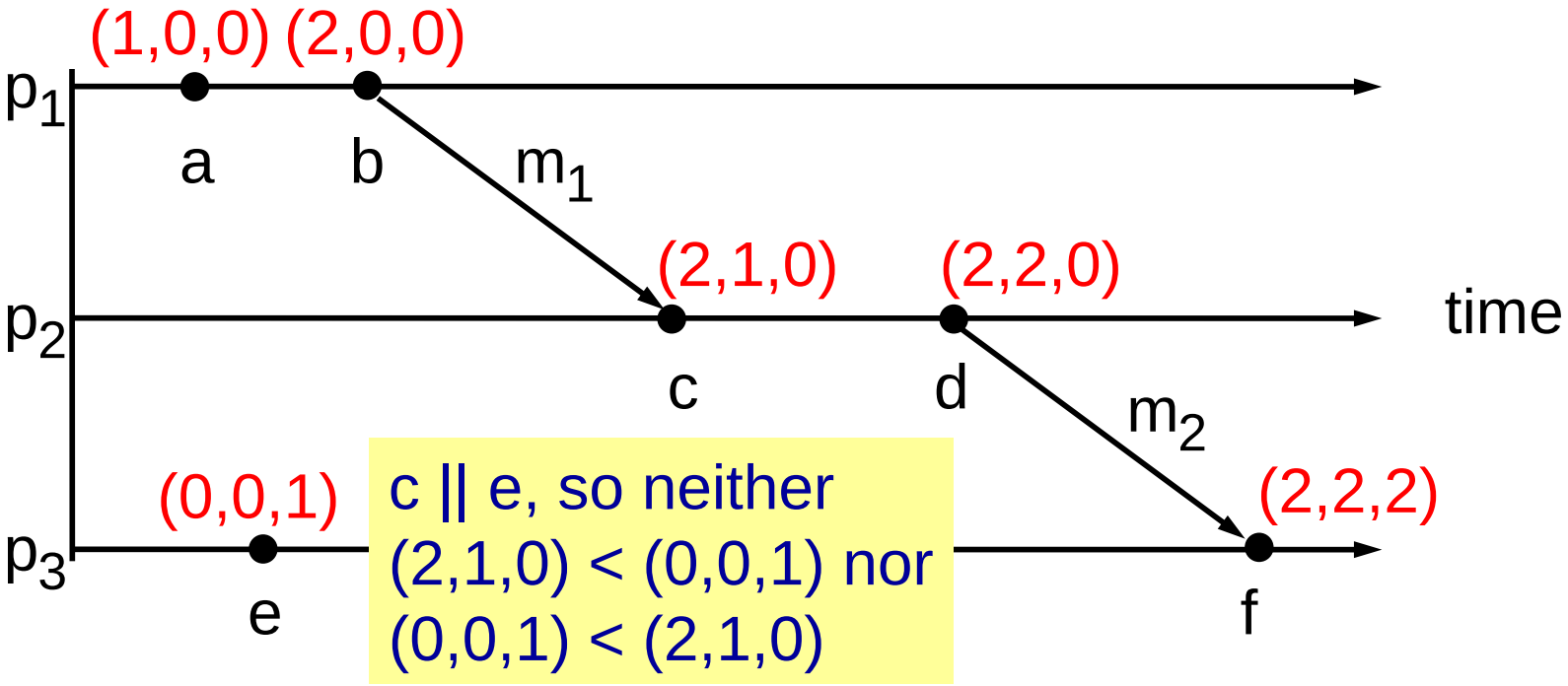
- Denote the timestamp of event e by $V(e)$
- If $e \rightarrow e'$, then $V(e) < V(e')$ (How to prove it?)

Example of Vector Clocks



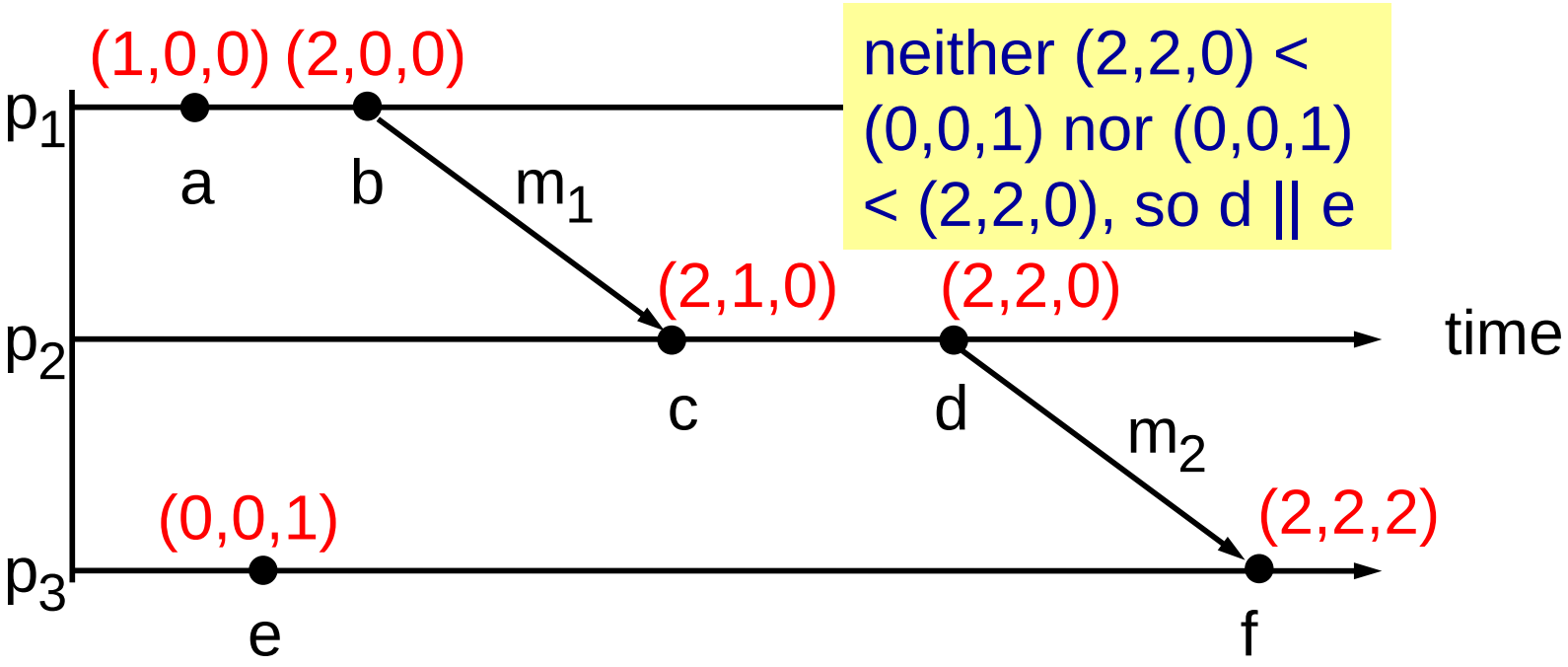
- If $V(e) < V(e')$, then $e \rightarrow e'$ (How to prove it?)

Example of Vector Clocks



- If $e \mid\mid e'$, then neither $V(e) \leq V(e')$ nor $V(e') \leq V(e)$
  – their timestamps are not comparable

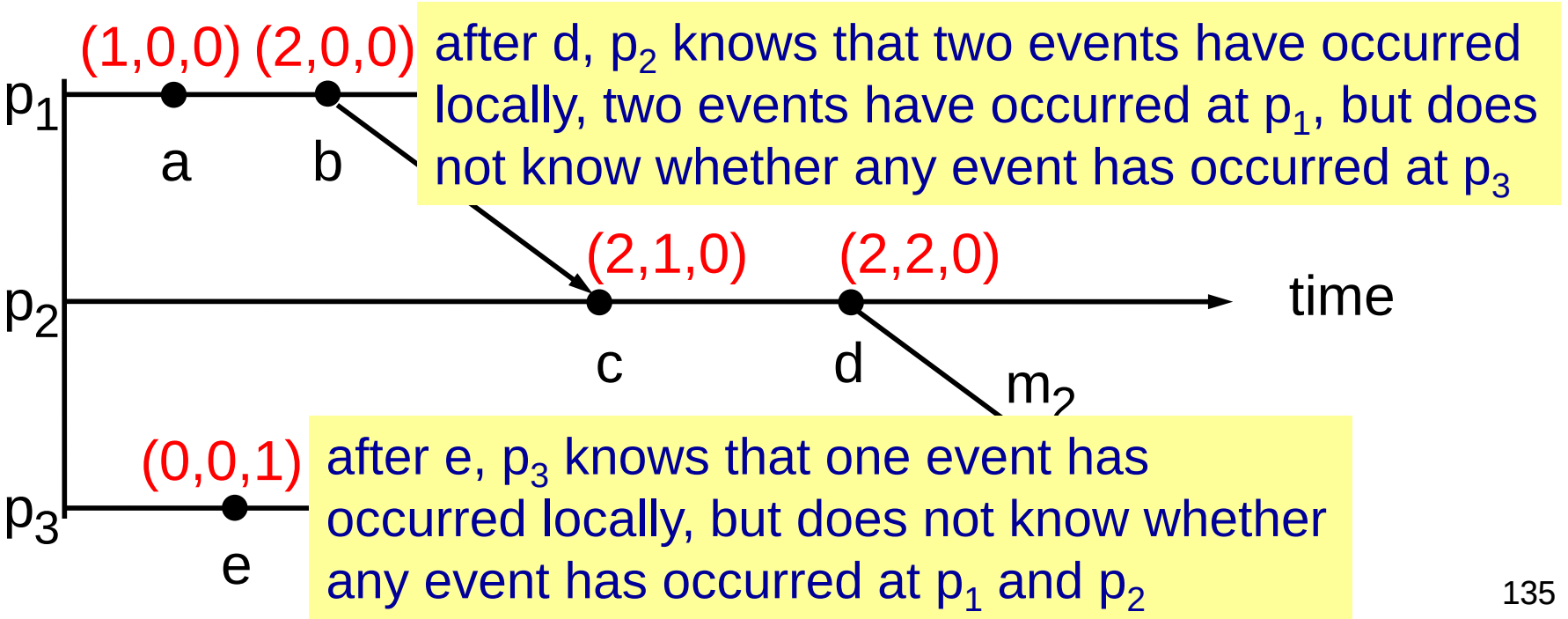
Example of Vector Clocks



- If neither $V(e) \leq V(e')$ nor $V(e') \leq V(e)$, then $e \mid\mid e'$

Vector Clocks

- Properties
  - For a vector clock V_i, $V_i[i]$ is the number of events that have occurred at p_i so far
  - For a vector clock V_i, $V_i[k]$ ($k \neq i$) is the number of events that have occurred at p_k that p_i knows



Prove if $V(e) < V(e')$, then $e \rightarrow e'$

- Given two events e and e', either (1) $e \rightarrow e'$; (2) $e' \rightarrow e$; or (3) e and e' are concurrent
- If $e' \rightarrow e$, then $V(e') < V(e)$
- So, to prove that if $V(e) < V(e')$, then $e \rightarrow e'$, we only need to prove that e and e' are not concurrent
- Now we are going to prove that if e and e' are concurrent, then neither $V(e) < V(e')$ nor $V(e') < V(e)$

Prove if $V(e) < V(e')$, then $e \rightarrow e'$

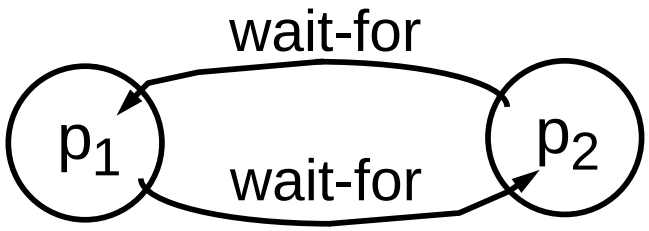
- If e and e' are concurrent, they occur at different processes
- Suppose e occurs at process p_i and e' occurs at process p_j
- According to vector clock mechanism, for any event x where $e \rightarrow x$, $V(e)[i] \leq V(x)[i]$; for all other events x, $V(x)[i] < V(e)[i]$
- Since e and e' are concurrent, $V(e')[i] < V(e)[i]$
- Similarly, $V(e)[j] < V(e')[j]$
- So, neither $V(e) < V(e')$ nor $V(e') < V(e)$

Outline

- Synchronizing Physical Clocks
- Causal Ordering and Logical Clocks
- Global States
- Distributed Debugging

Motivation

- In many situations, we need to find out whether a particular property is true when a distributed system executes
- Example: distributed deadlock detection



  - A distributed deadlock occurs when
    - Each of a collection of processes waits for another process to send it a message
    - There is a cycle in the graph of this 'wait-for' relationship

System Model

- Without global time, can we assemble a meaningful global state from local states recorded by different processes at different real times?
- History of process p_i
  - p_i's history: all events that take place within p_i and ordered by their occurrences

$$h_i = < e_i^1, e_i^2, \ldots >$$

  - Prefix of p_i's history containing the first k events:

$$h_i^k = <e_i^1, e_i^2, \ldots, e_i^k>$$

System Model

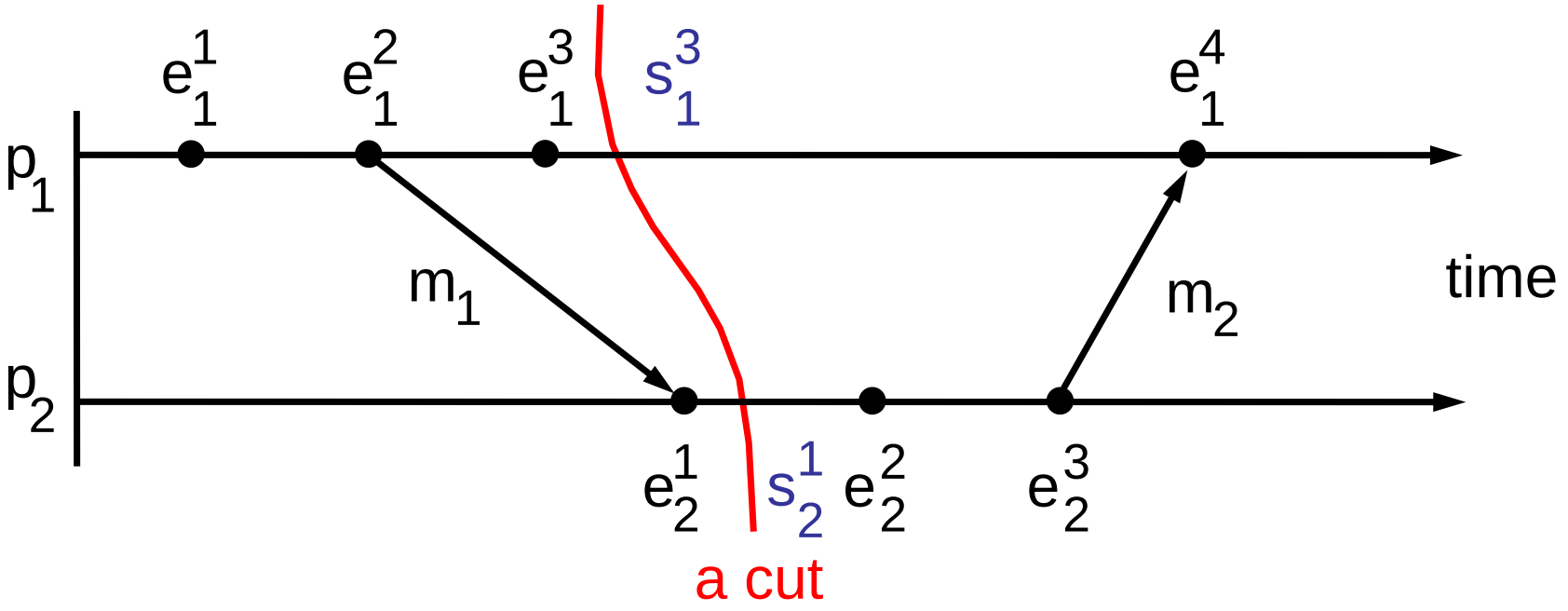
- Cut
  - A union of the prefixes of process histories:

$$C = h_1^{c_1} \bigcup h_2^{c_2} \bigcup ... \bigcup h_N^{c_N}$$

- Global state
  - A set of the states of all processes
    - We refer to the state of process p_i immediately after its kth event occurs as s_i^k (s_i^0 denotes the initial state of p_i)
  - Each global state corresponds to a cut – the set of the process states immediately after the occurrence of the last event of each process in the cut:

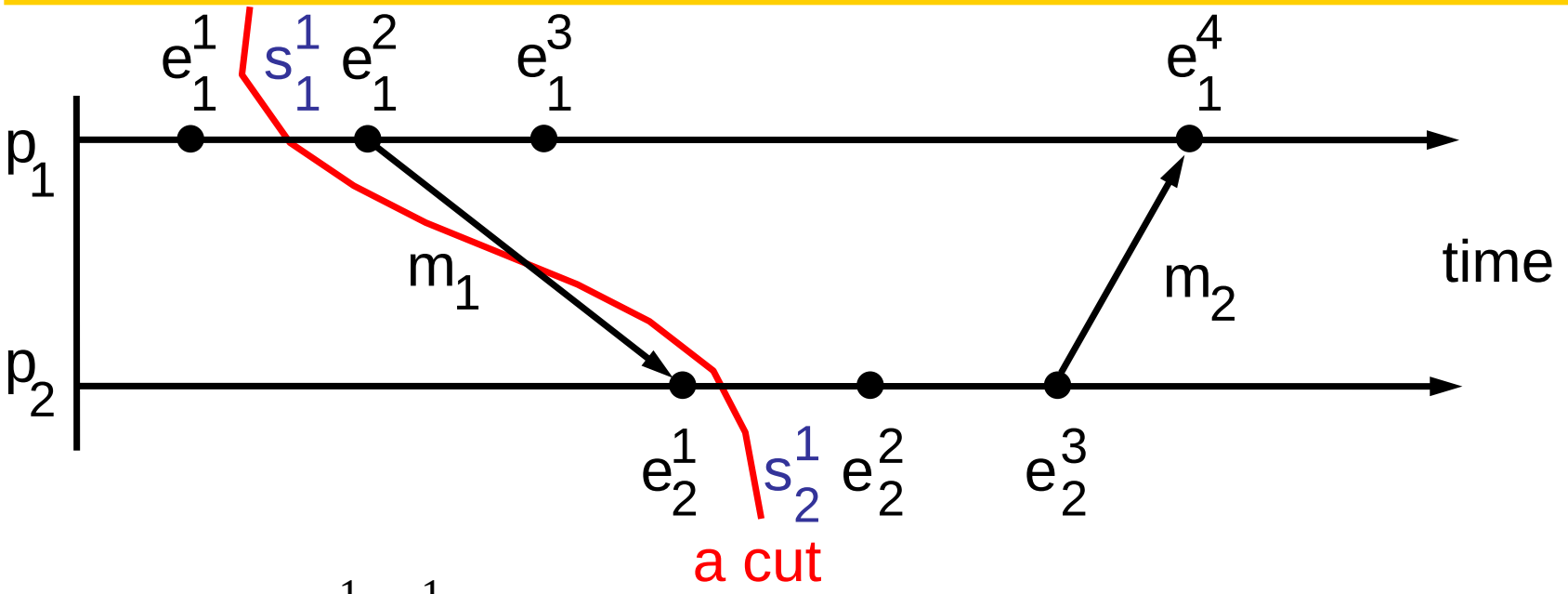
$$< s_1^{c_1}, s_2^{c_2}, ..., s_N^{c_N} >$$

Example of Cut



- Cut: $< e_1^1, e_1^2, e_1^3, e_2^1 >$
- Global state corresponding to the cut: $< s_1^3, s_2^1 >$

Example of Cut

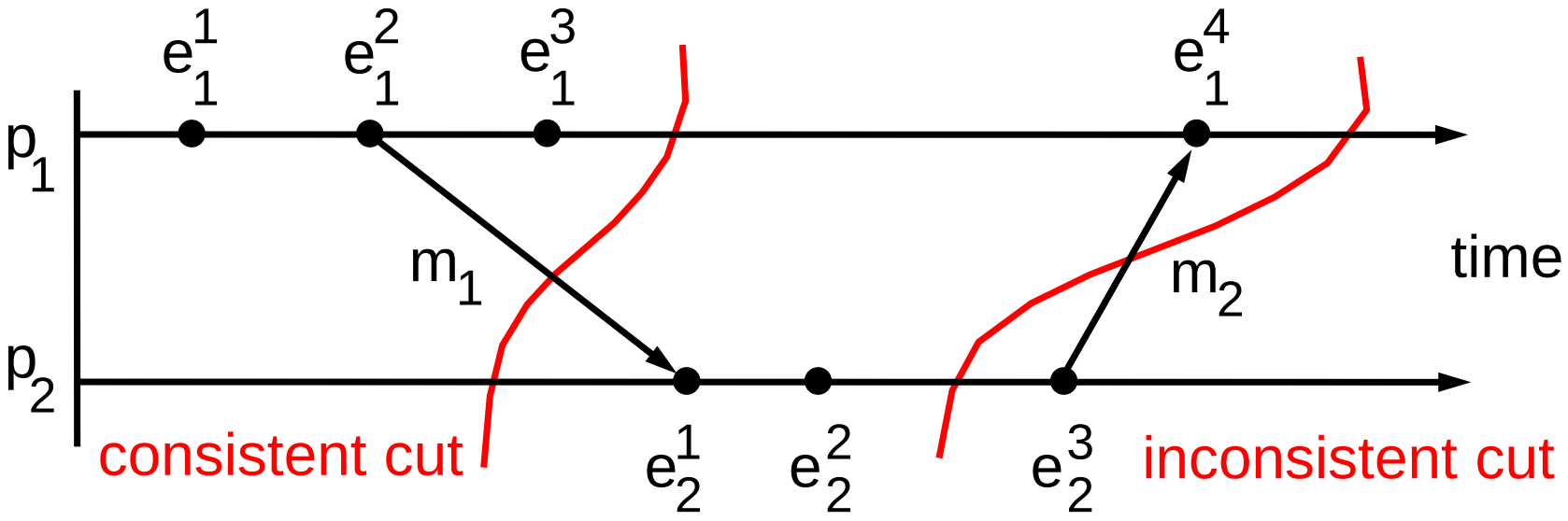


- Cut: $< e_1^1, e_2^1 >$
- Global state corresponding to the cut: $< s_1^1, s_2^1 >$
- However, are the processes likely to be in these states at the same time in actual execution? – No, because p_2 cannot have received m_1 before p_1 sends it

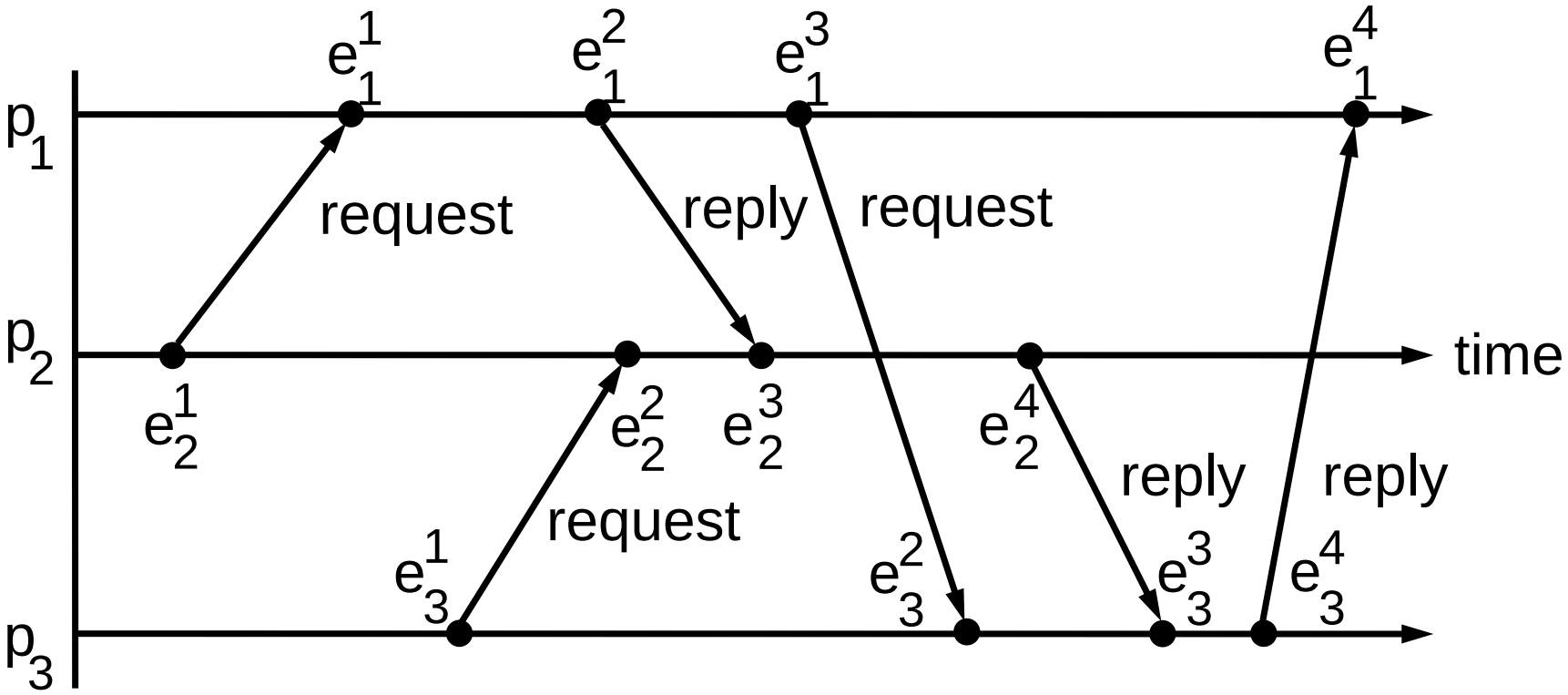
Consistent Cut

- We are interested in the cuts that are consistent with the progress of events
- Consistent cut
  - A cut C is consistent if, for all events $e \in C$, $f \rightarrow e \Rightarrow f \in C$
  - Otherwise, the cut is known as inconsistent
- Consistent global state
  - A consistent global state corresponds to a consistent cut
  - An inconsistent global state corresponds to an inconsistent cut
  - In actual execution, a system passes through consistent global states only

Example of Consistent Cut

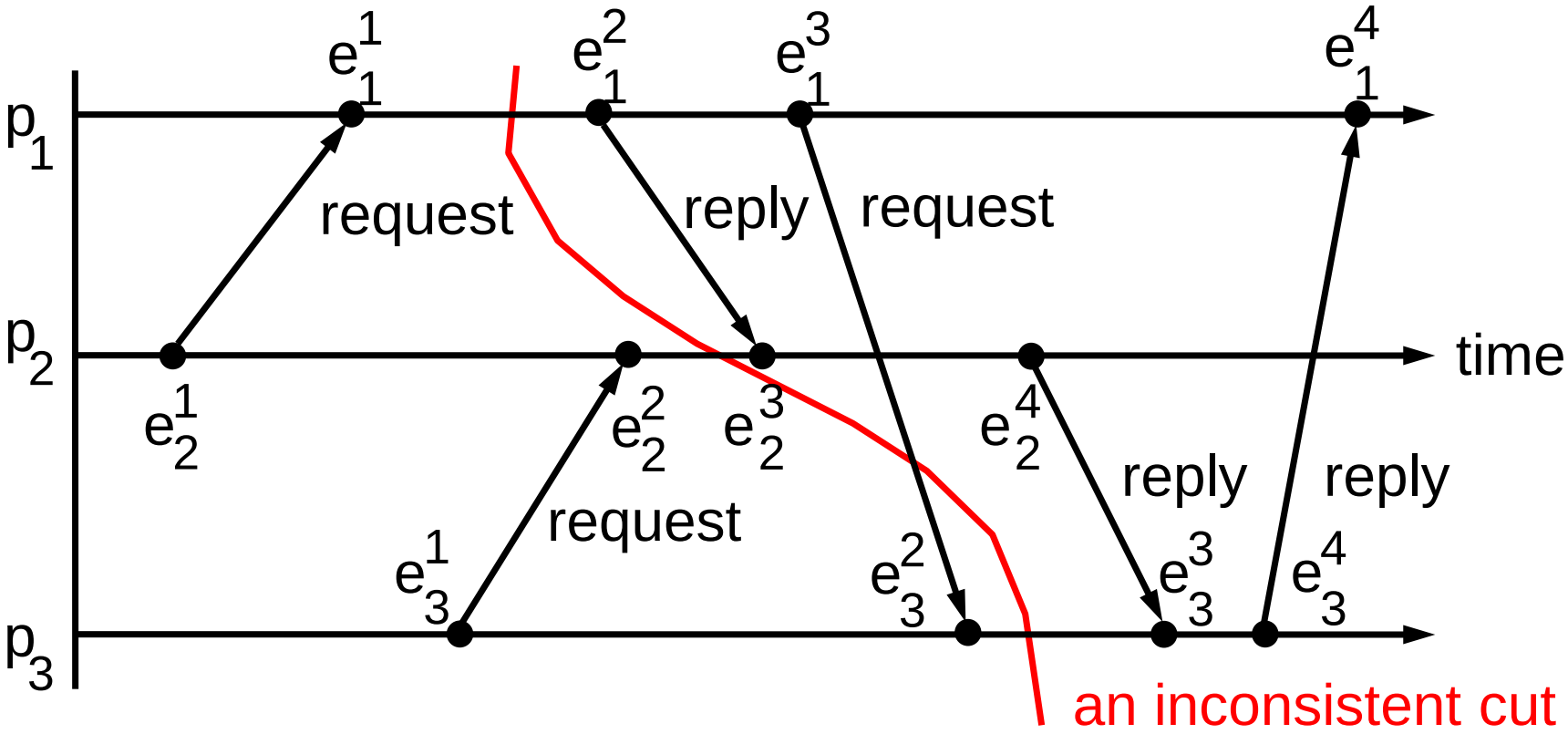


Why Is Consistent Cut Important?



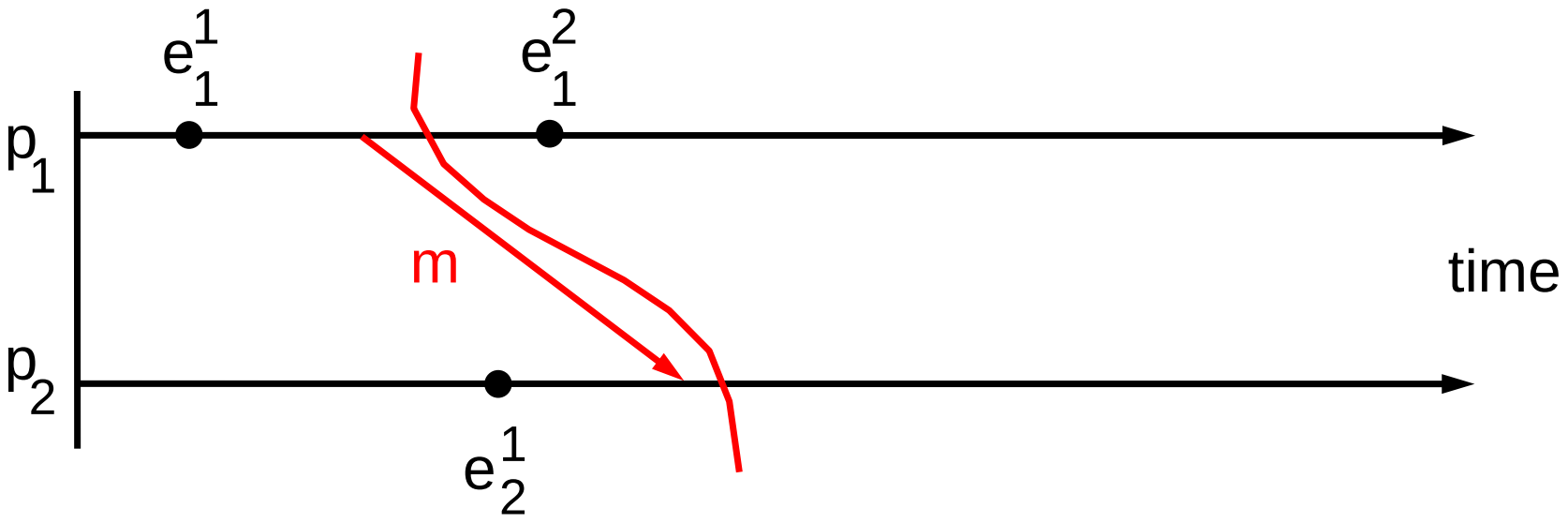
- Each process either waits for requests from other processes and handles them or sends a request to another process and waits for reply
- There is no deadlock in the above execution

Why Is Consistent Cut Important?



- However, if someone derives process states from an inconsistent cut, he may conclude that there is a deadlock because p_1 is waiting for p_3, p_2 is waiting for p_1, and p_3 is waiting for p_2

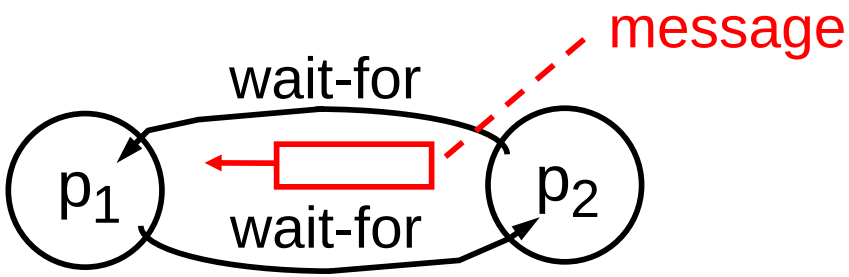
How to Record A Snapshot?



- p_1 records its local state and then sends a marker message m to p_2 who records its local state as soon as it receives the message
- Good enough? – in many applications, the state of communication channels is also relevant

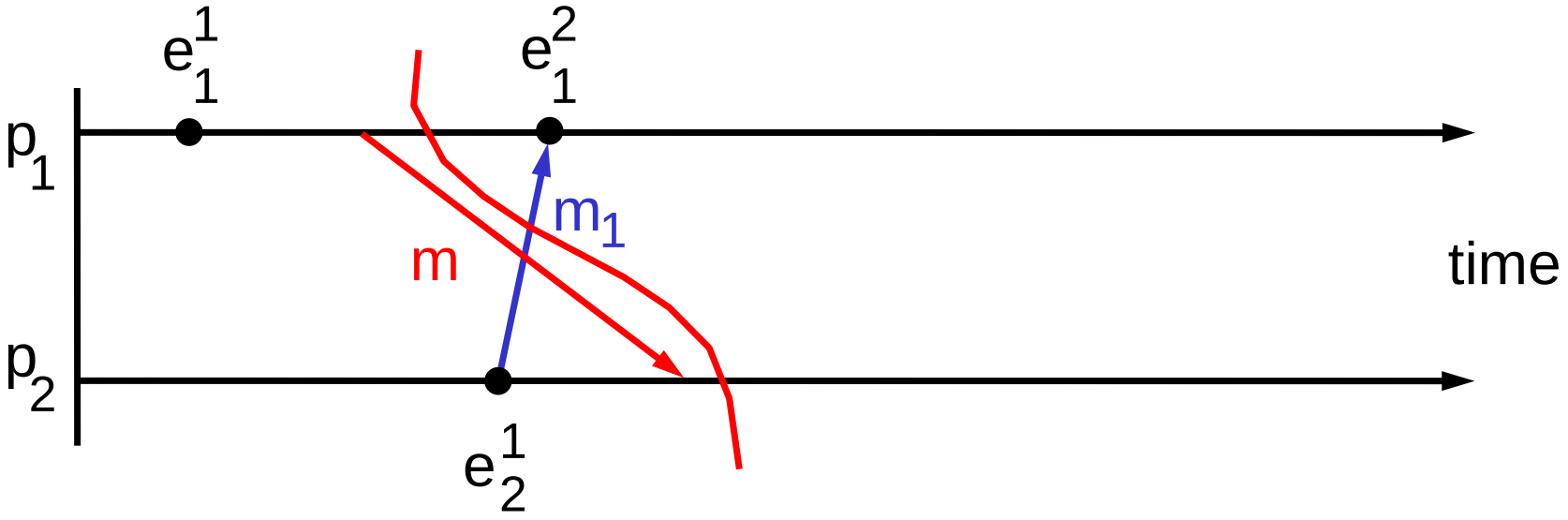
How to Record A Snapshot?

- Revisit example: distributed deadlock detection



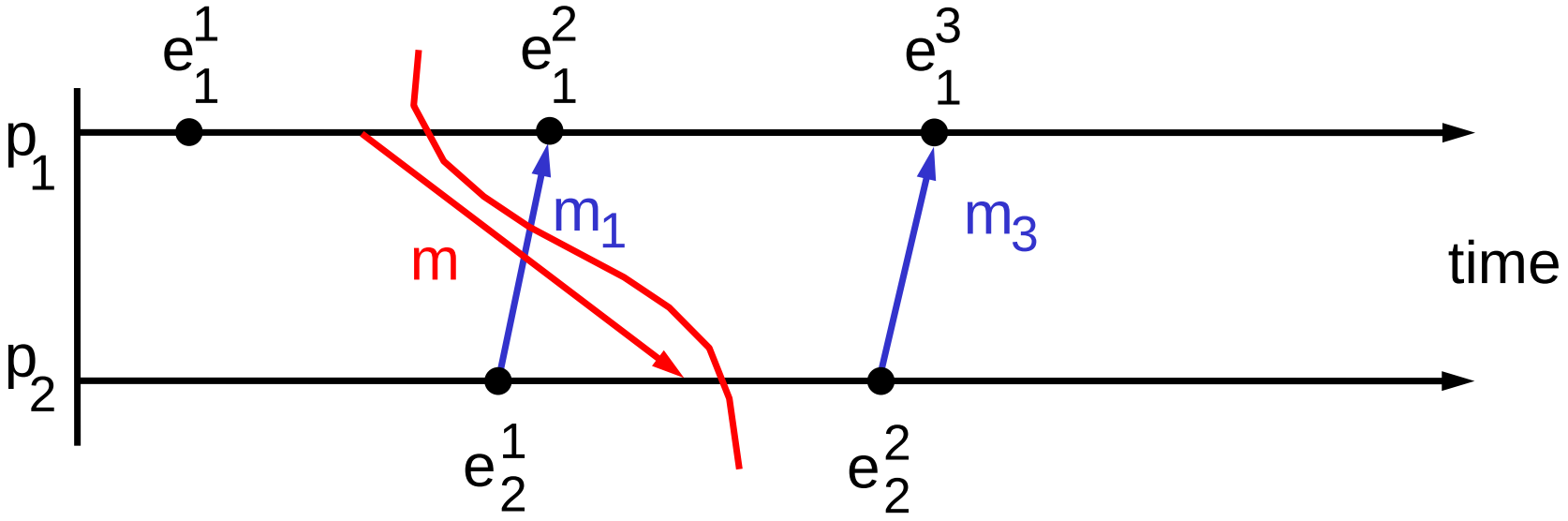
  - If there is a message in transition at this time, there may not be a deadlock
    - For example, p_1 is a server and p_2 is a client, p_1 is waiting for the request, p_2 just sends out a request and is waiting for the reply.
  - So, we must include the state of communication channels as well as the state of processes in the snapshot

How to Record A Snapshot?



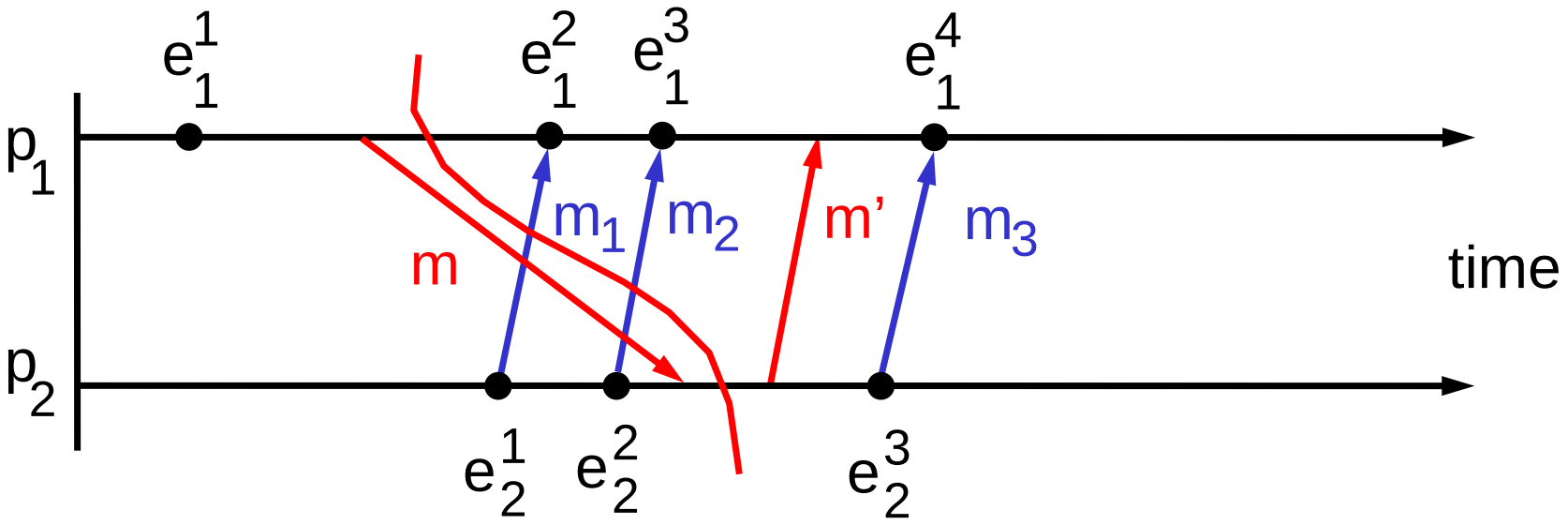
- What if p_2 also sends a message to p_1?
- We also need to record the state of the channel
- So, p_1 needs to record the messages (e.g., m_1) from p_2 after it sends m to p_2

How to Record A Snapshot?



- But how does p_1 determine when to stop recording?
- Should p_1 record m_3 as well? – No

How to Record A Snapshot?



- On receiving m, before sending any other messages, p_2 sends a marker message m' to p_1 who uses it to finish recording channel states
- p_1 should record all messages between sending of m and receipt of m' only (i.e., m_1 and m_2)
- Now, let's extend this idea to N processes

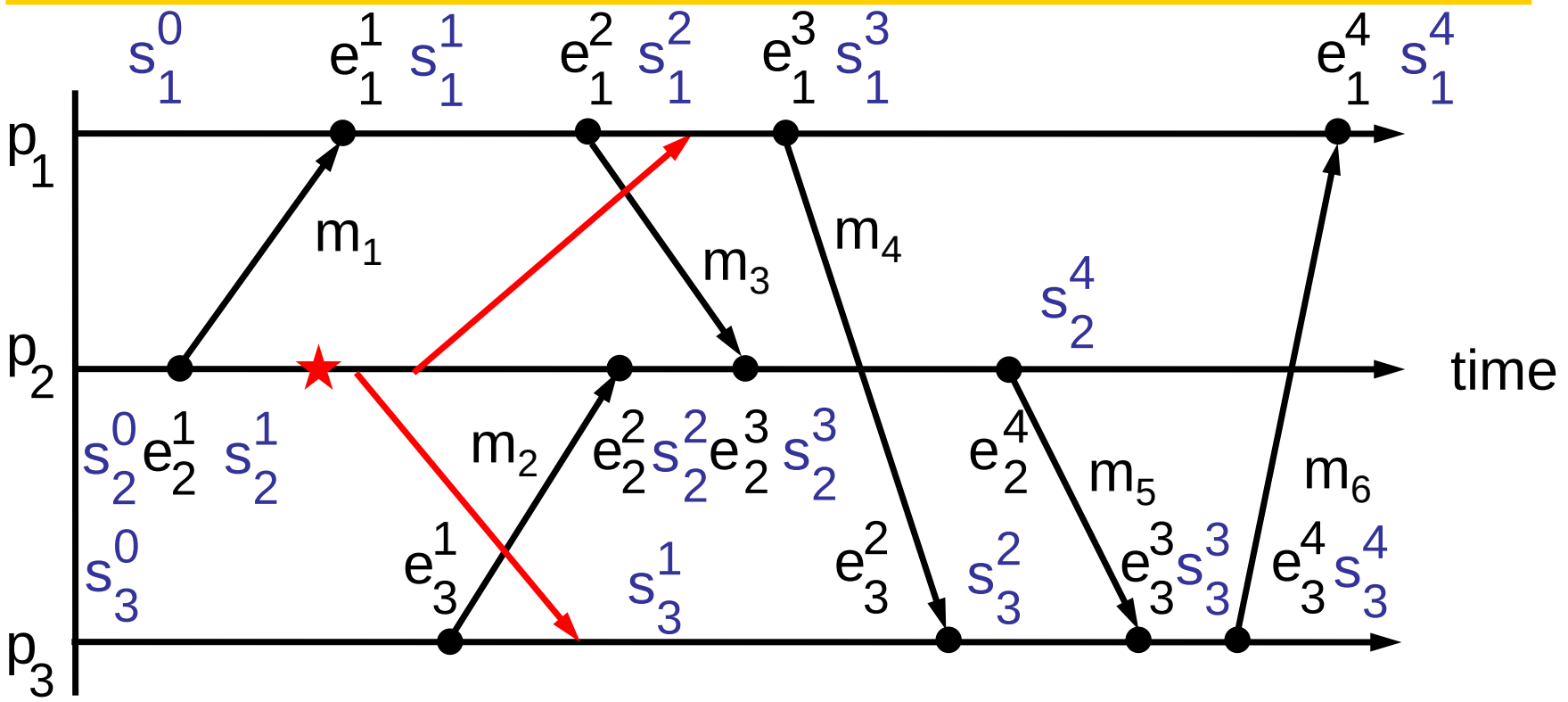
Chandy and Lamport Algorithm

- Objective: to record a snapshot of a distributed system during execution
  - Any process may initiate a snapshot recording at any time
- Assumptions
  - Processes are connected to each other through unidirectional point-to-point channels
    - N processes → N (N – 1) channels
    - For each process, we shall distinguish its incoming channels and outgoing channels
  - Neither channels nor processes fail
  - Reliable communication (validity and integrity)
  - FIFO-ordered message delivery on each channel

Chandy and Lamport Algorithm

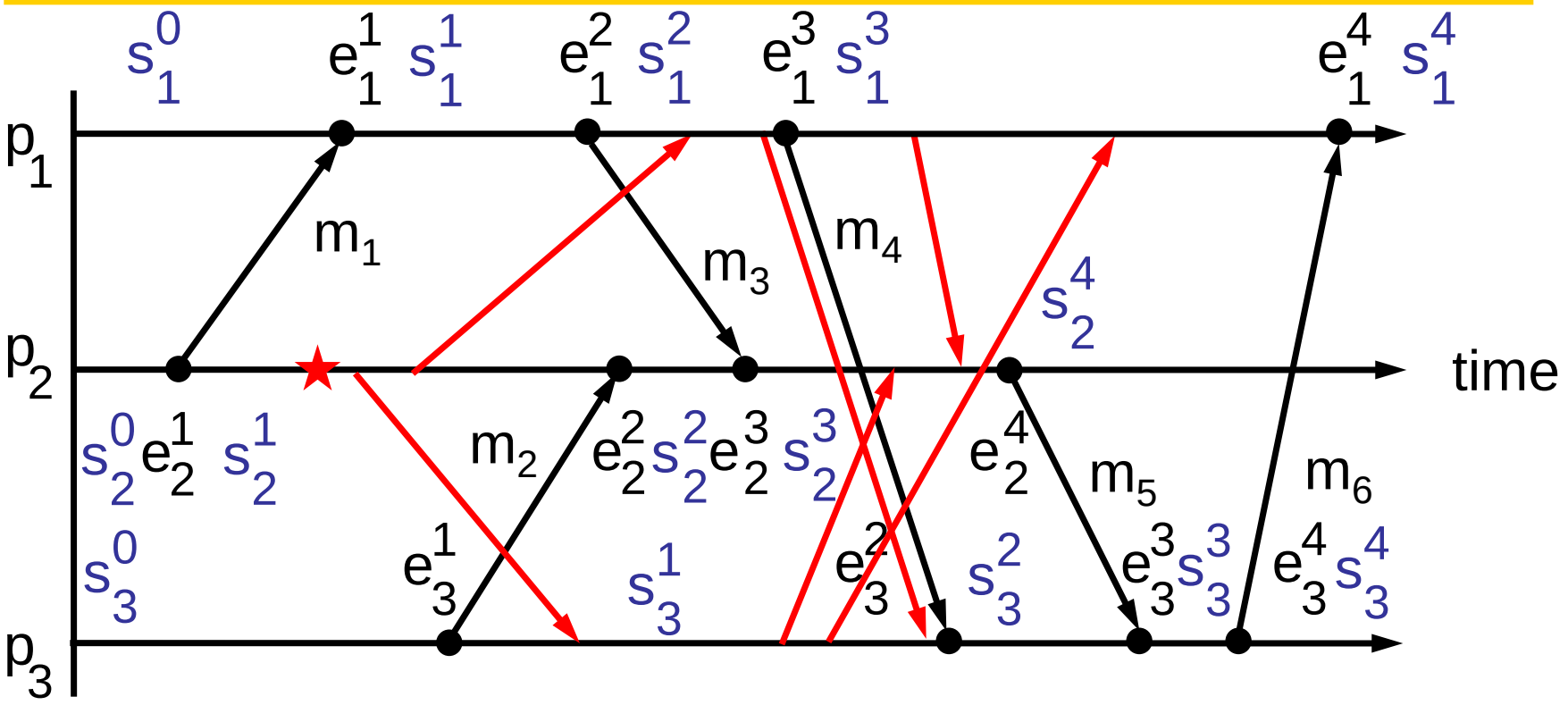
- Approach: use marker messages
  - As a prompt for the receiver to save its own local state if it has not already done so
  - As a means of determining which messages to include in the channel state

Example



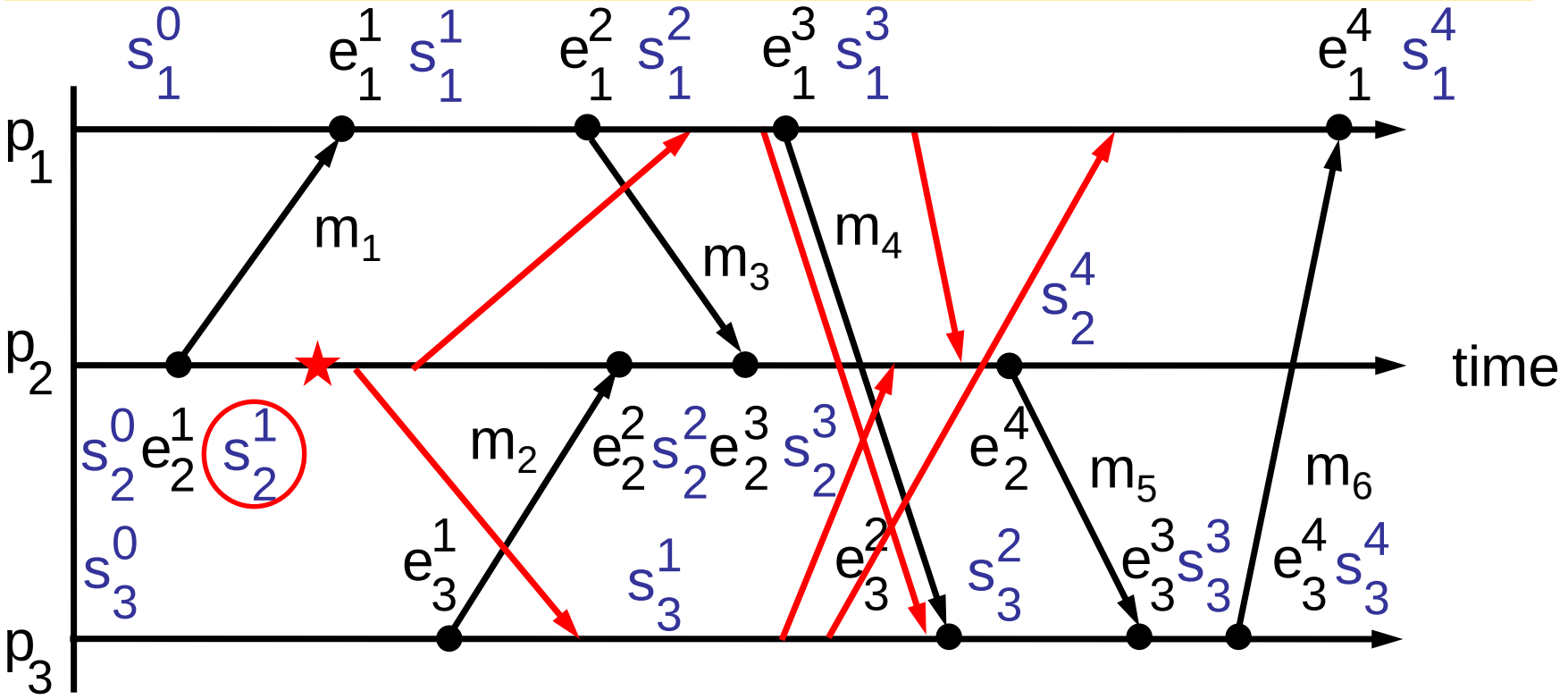
- p_2 initiates a snapshot recording at ★ between e_2^1 and e_2^2
- The initiating process sends a marker message along each outgoing channel to all other processes before it sends any other application message along that channel

Example



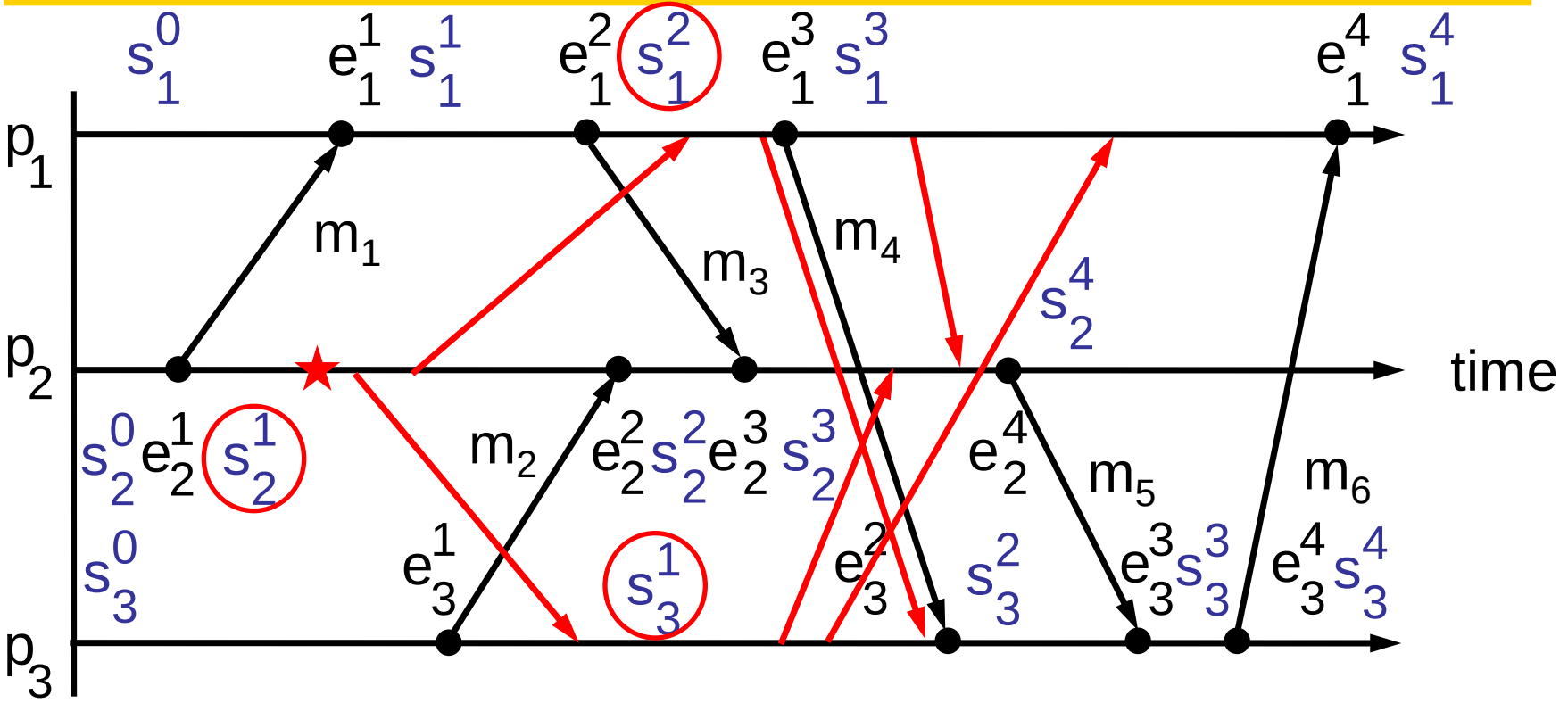
- On receiving the first marker message, each non-initiating process sends a marker message along each outgoing channel to all other processes before it sends any other application message along that channel

Example



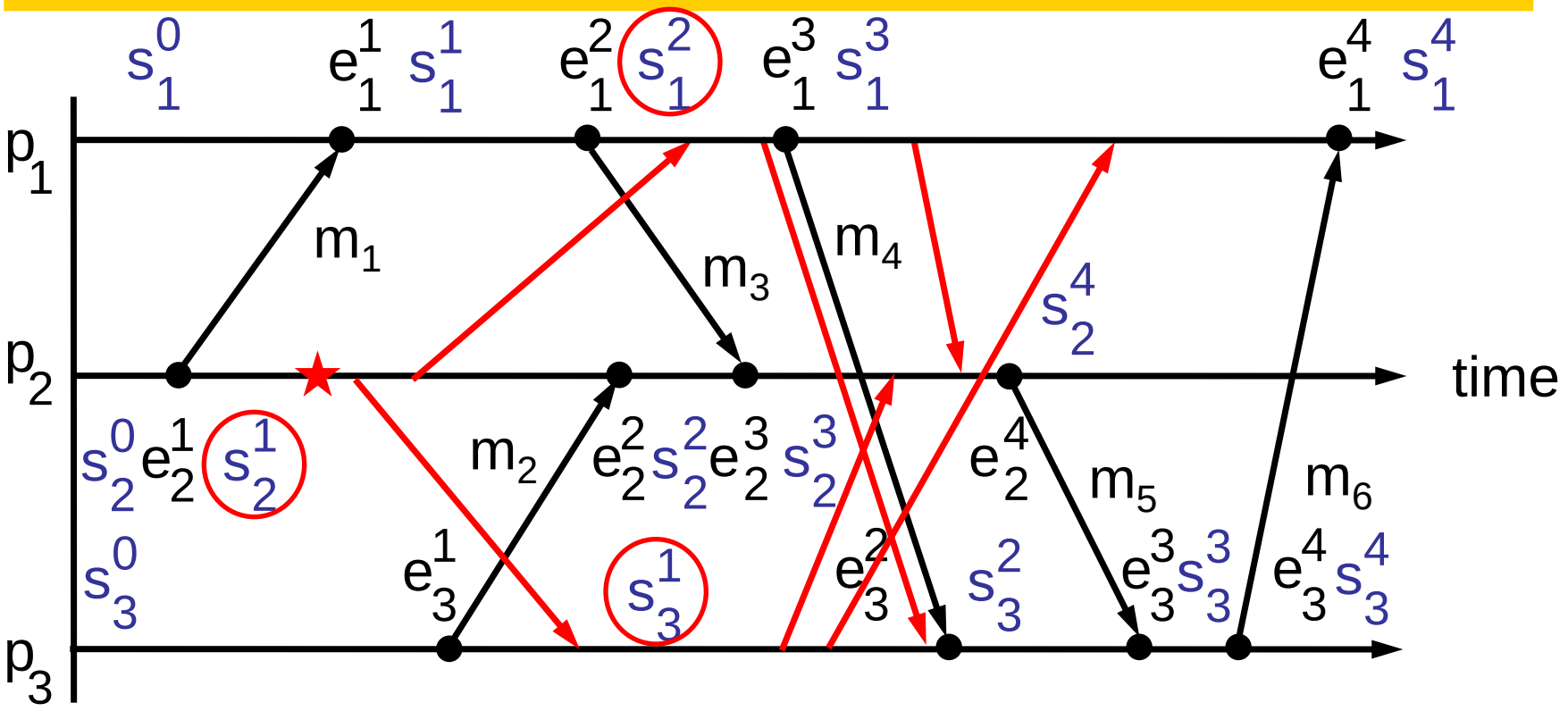
- p_2 initiates a snapshot recording at ★ between e_2^1 and e_2^2
- The initiating process records its process state before sending out the marker messages

Example



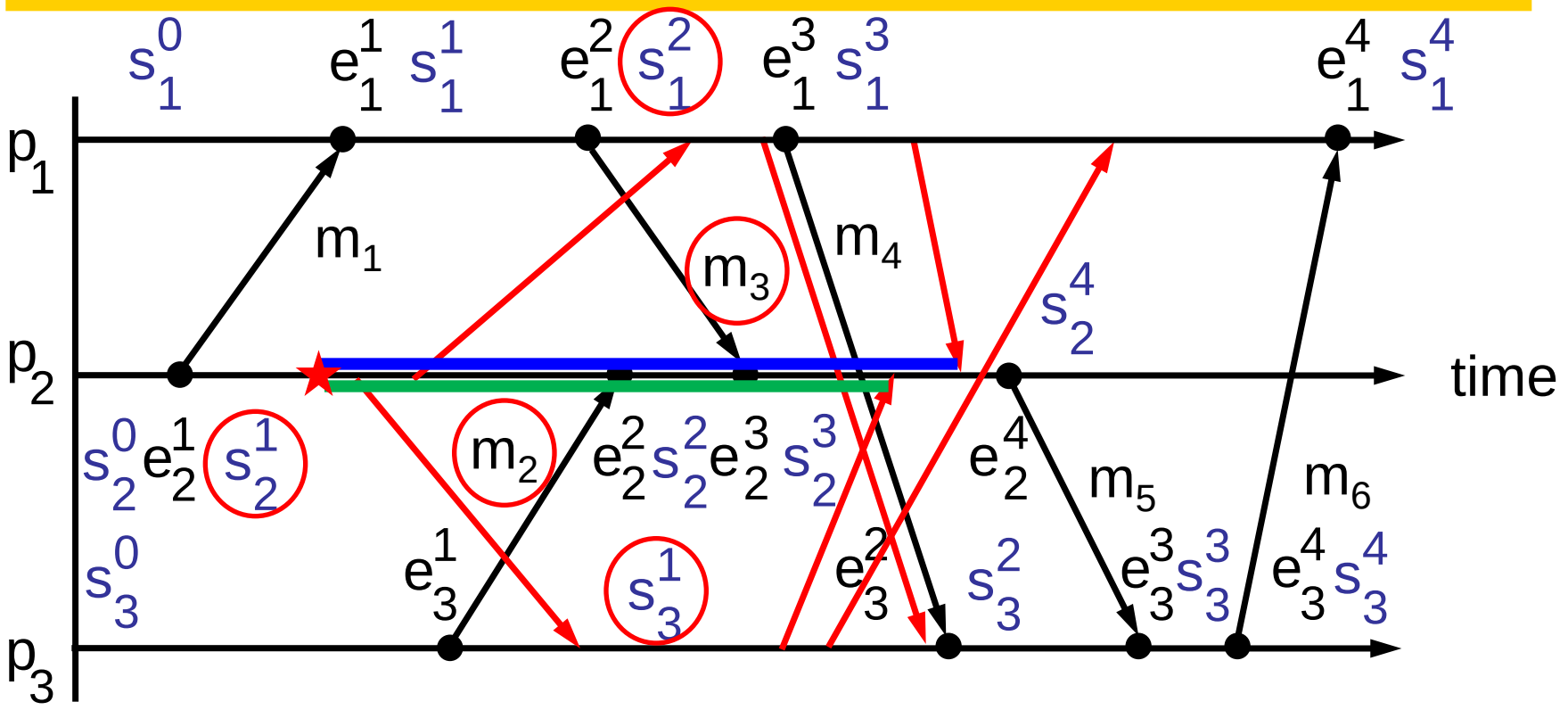
- p_2 initiates a snapshot recording at ★ between e_2^1 and e_2^2
- Each non-initiating process records its process state on receiving the first marker message

Example



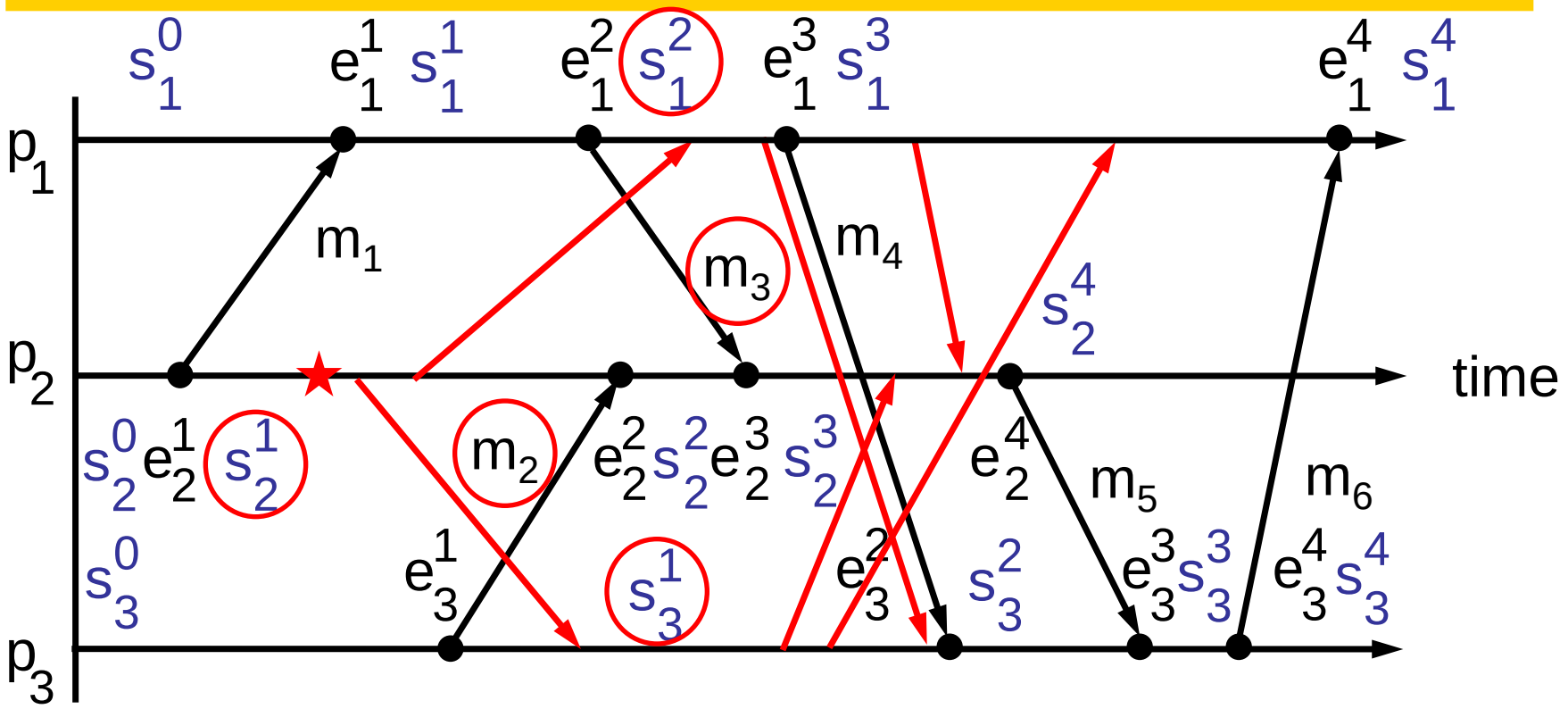
- Denote the unidirectional channel from process p_i to process p_j by $c_{i \to j}$
- Channel $c_{i \to j}$'s state is recorded by process p_j

Example



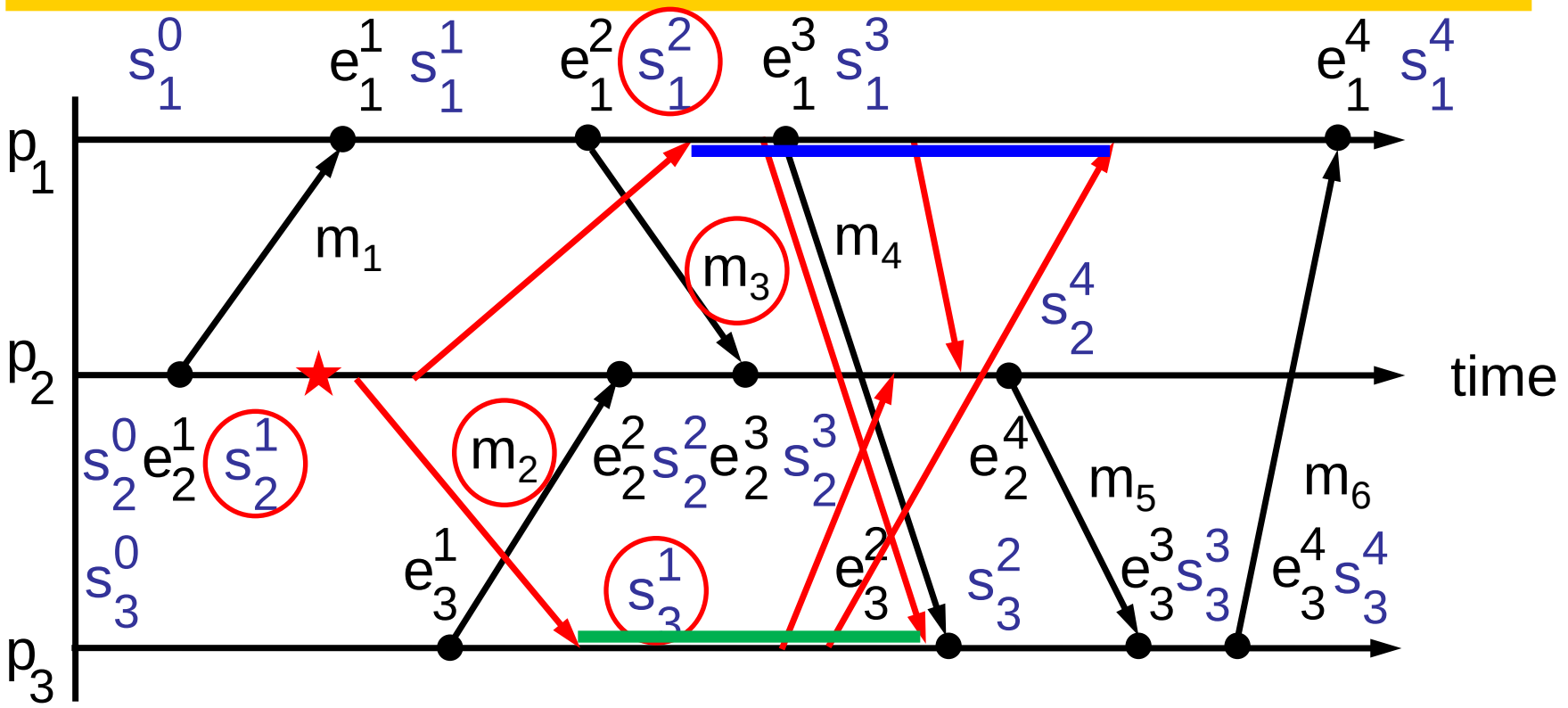
- If p_j is the initiating process, p_j records the messages arriving on channel $c_{i \to j}$ between p_j initiates the snapshot recording and receives the marker message from p_i
- So, $c_{1 \to 2}$ state = $<m_3>$, $c_{3 \to 2}$ state = $<m_2>$

Example



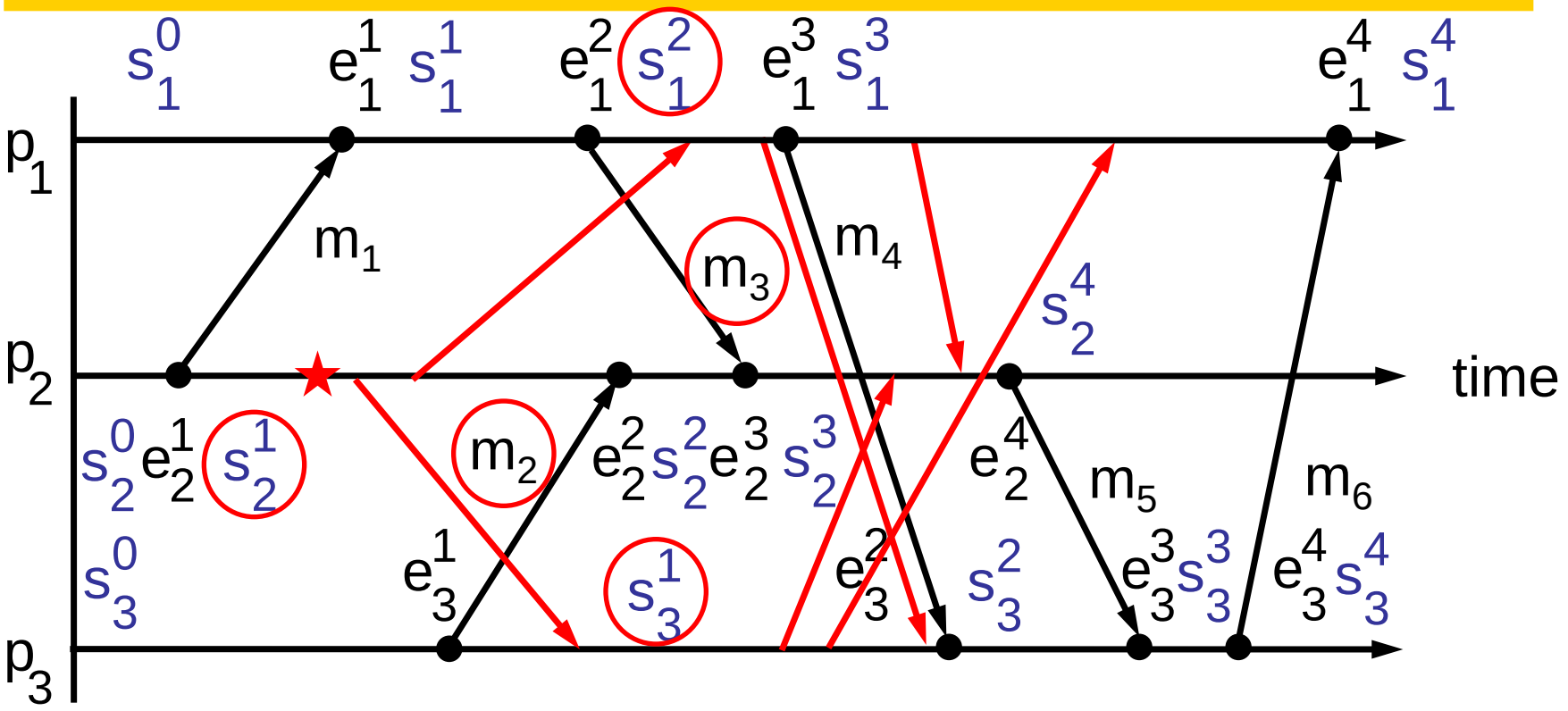
- If p_j is a non-initiating process and $c_{i\to j}$ is the channel through which p_j receives its first marker message, $c_{i\to j}$'s state is recorded as an empty set
- So, $c_{2\to 1}$ state = < >, $c_{2\to 3}$ state = < >

Example



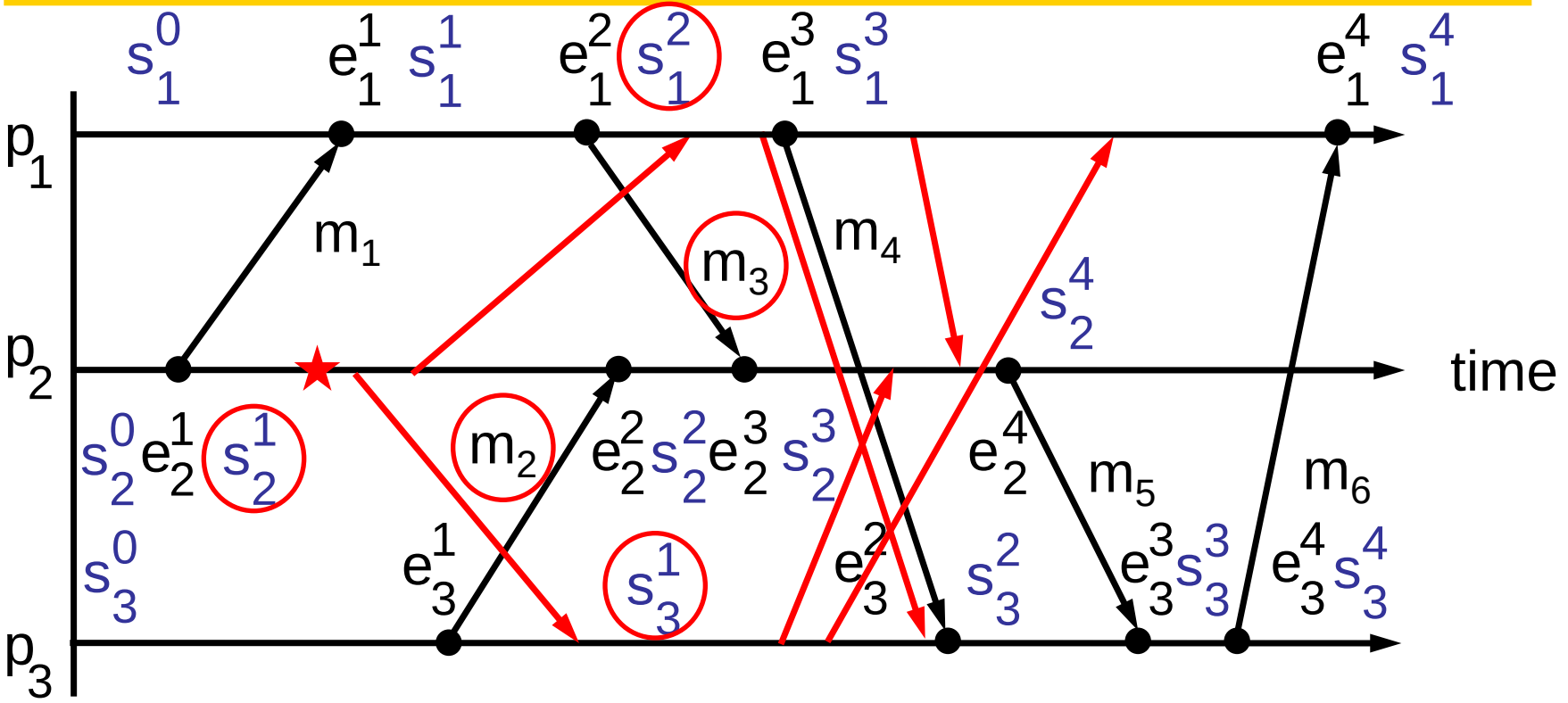
- If p_j is a non-initiating process and $c_{i\to j}$ is not the channel through which p_j receives its first marker message, p_j records the messages arriving on channel $c_{i\to j}$ between p_j receives the first marker message and the marker message from p_i
- So, $c_{1\to 3}$ state = < >, $c_{3\to 1}$ state = < >

Example



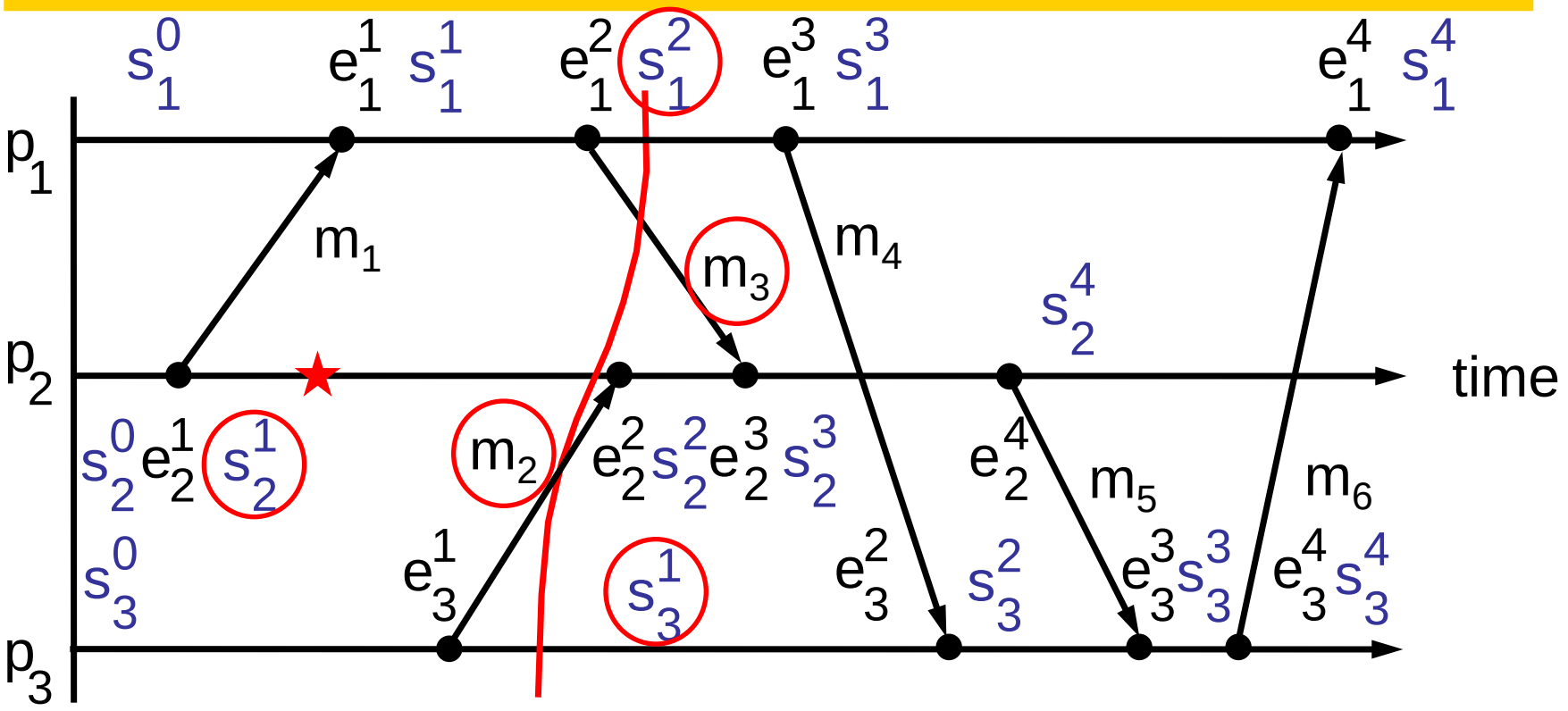
- Each process completes all recording activities when it has received a marker message from all other processes
- Then, it may send the recorded states (local process state + channel states) to the initiating process

Example



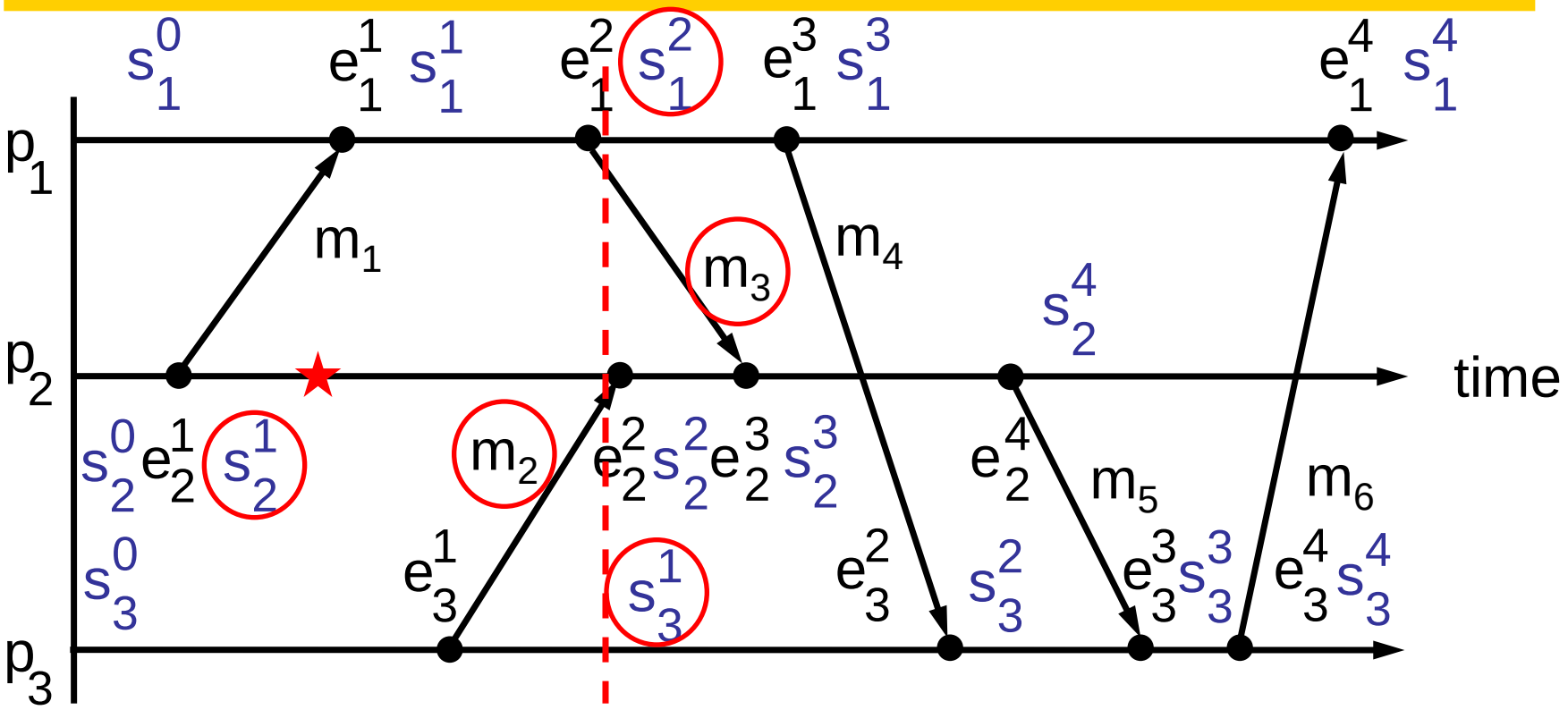
- Final recorded snapshot: p_1 state = s_1^2, p_2 state = s_2^1, p_3 state = s_3^1, $c_{1\to2}$ state = $\langle m_3 \rangle$, $c_{2\to1}$ state = $\langle\,\rangle$, $c_{1\to3}$ state = $\langle\,\rangle$, $c_{3\to1}$ state = $\langle\,\rangle$, $c_{2\to3}$ state = $\langle\,\rangle$, $c_{3\to2}$ state = $\langle m_2 \rangle$

Example



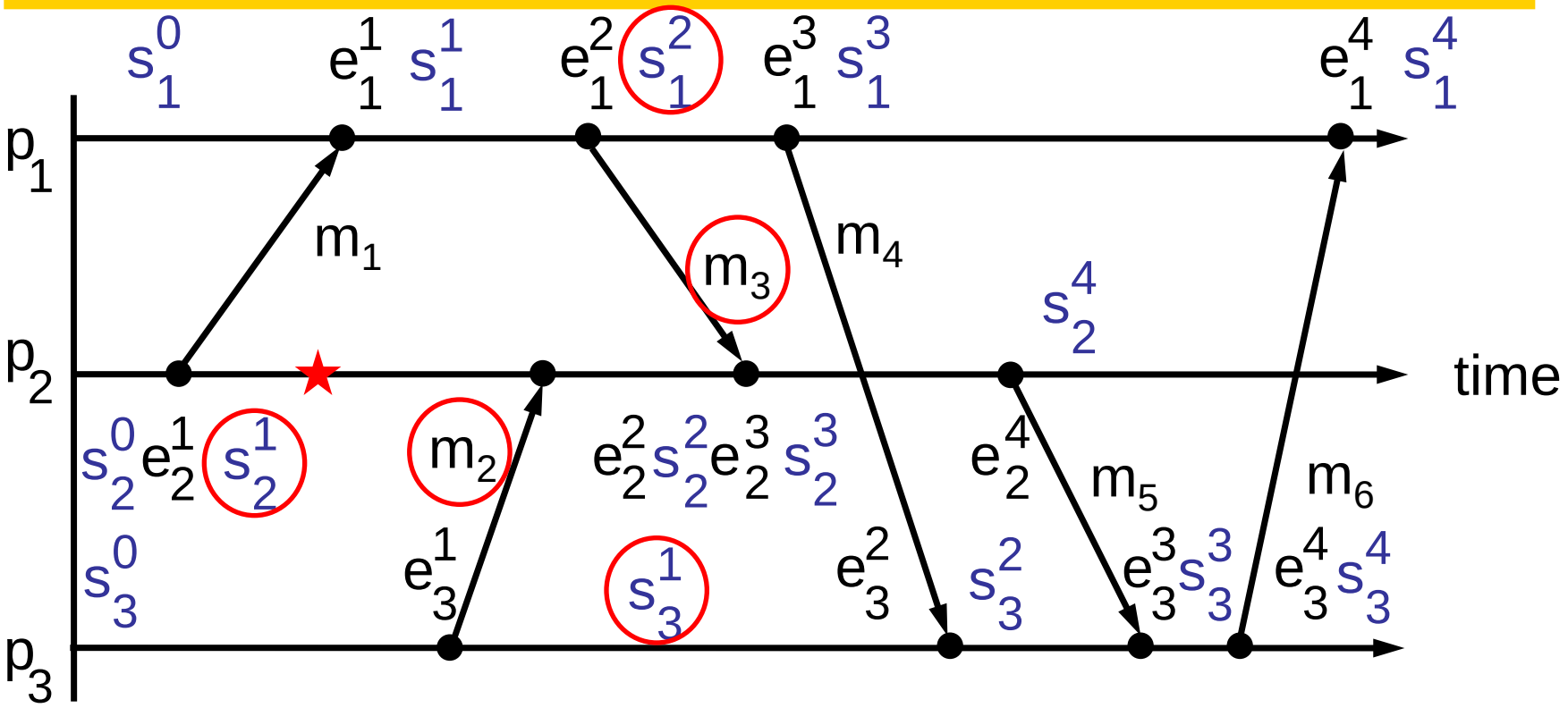
- Chandy and Lamport algorithm always selects a consistent cut
  - Proof omitted

Example



- However, the states recorded by Chandy and Lamport algorithm may not have occurred at the same time

Example



- There is no timepoint at which all recorded states occur

Outline

- Synchronizing Physical Clocks
- Causal Ordering and Logical Clocks
- Global States
- Distributed Debugging

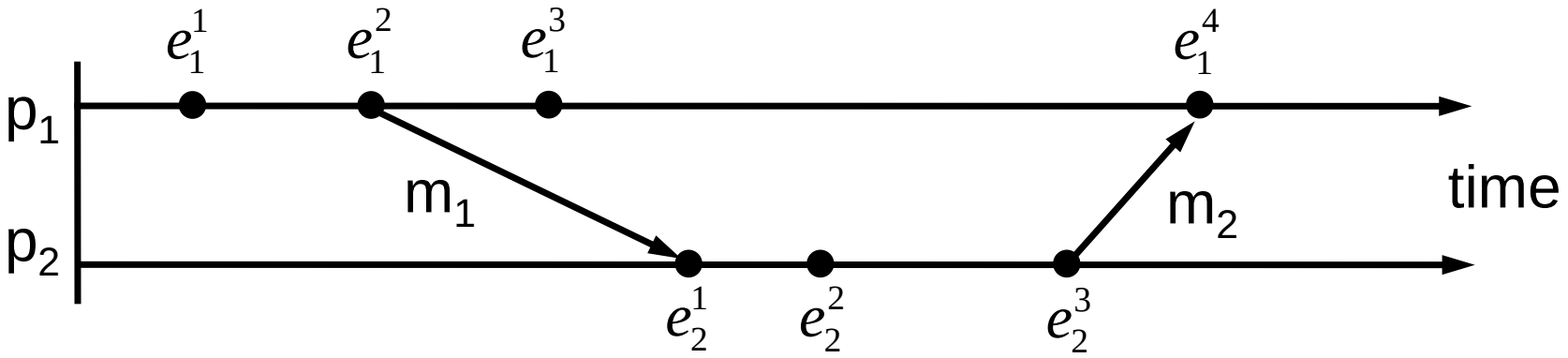
Distributed Debugging

- To establish what occurred during the execution of a distributed system and to check whether they are correct
- Example
  - An application: each process p_i contains a variable x_i
  - The variables change as the application executes
  - They are always required to be within a value δ of one another, i.e., $|x_i - x_j| \leq \delta$ for all i and j
  - How to check whether the constraints were broken during the execution? – must find out the values of variables that occur at the same time and evaluate their relationship

Distributed Debugging

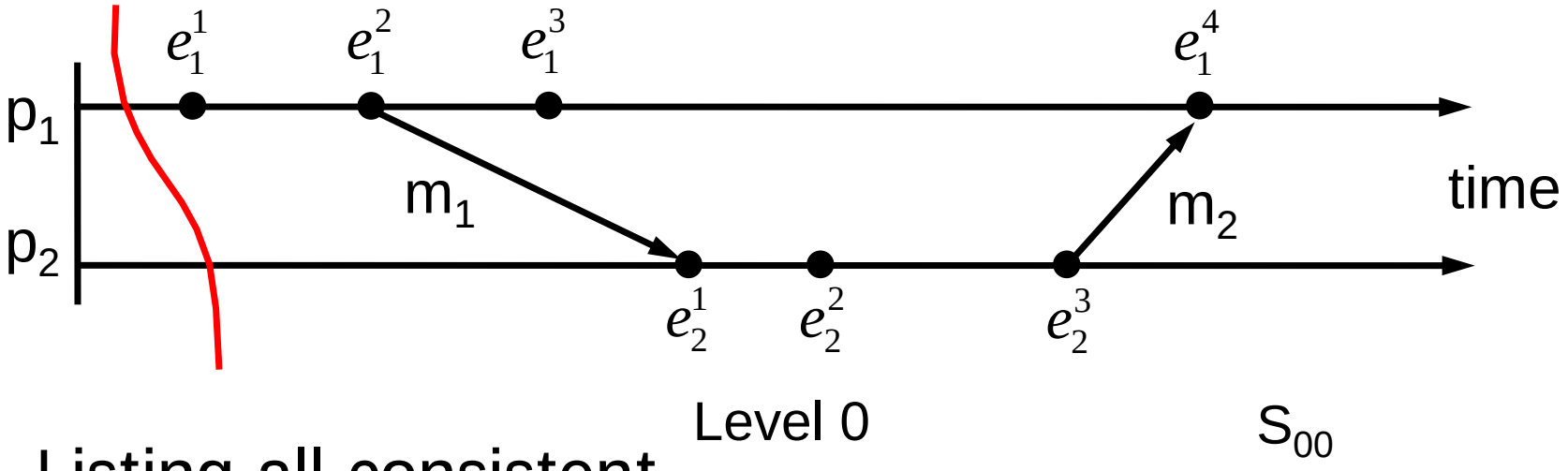
- Without global time, how to check?
- Can we use Chandy and Lamport algorithm to repeatedly record snapshots?
  - High communication overhead
  - Cannot continuously and completely "trace" global states over time
- Our approach: enumerate all consistent global states and check them exhaustively

Lattice of Consistent Global States



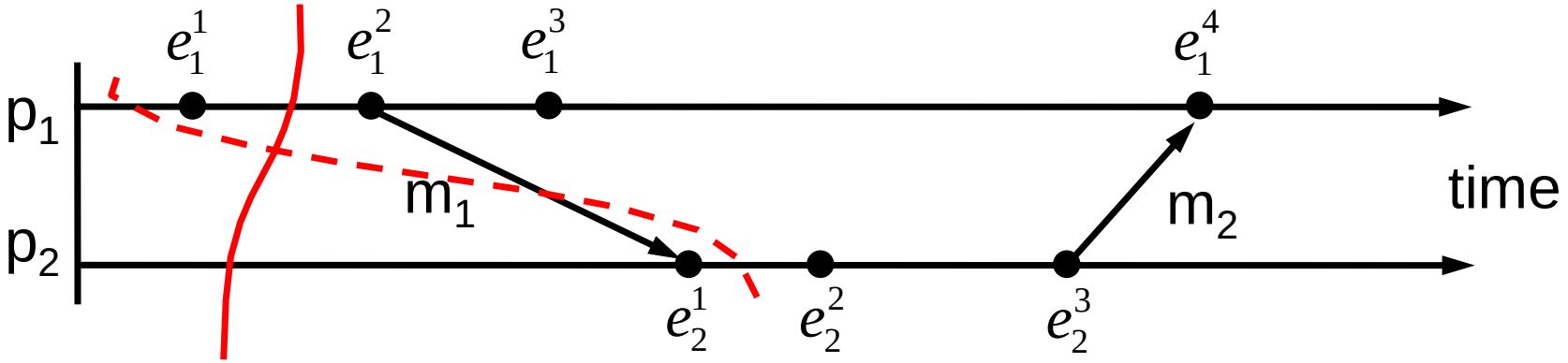
- Listing all consistent global states through a lattice
  - Starting from the initial state, advance the global state by one event at a time
  - A global state reachable from state $< s_1^{c_1}, s_2^{c_2}, ..., s_N^{c_N} >$ has the form of $< s_1^{c_1}, s_2^{c_2}, ..., s_i^{c_i+1}, \ldots, s_N^{c_N} >$

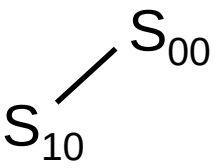
Lattice of Consistent Global States



- Listing all consistent global states through a lattice
  - $S_{ij} = < s_1^i, s_2^j >$: global state after i events at process p_1 and j events at process p_2
  - Initial global state is S_{00}

Lattice of Consistent Global States



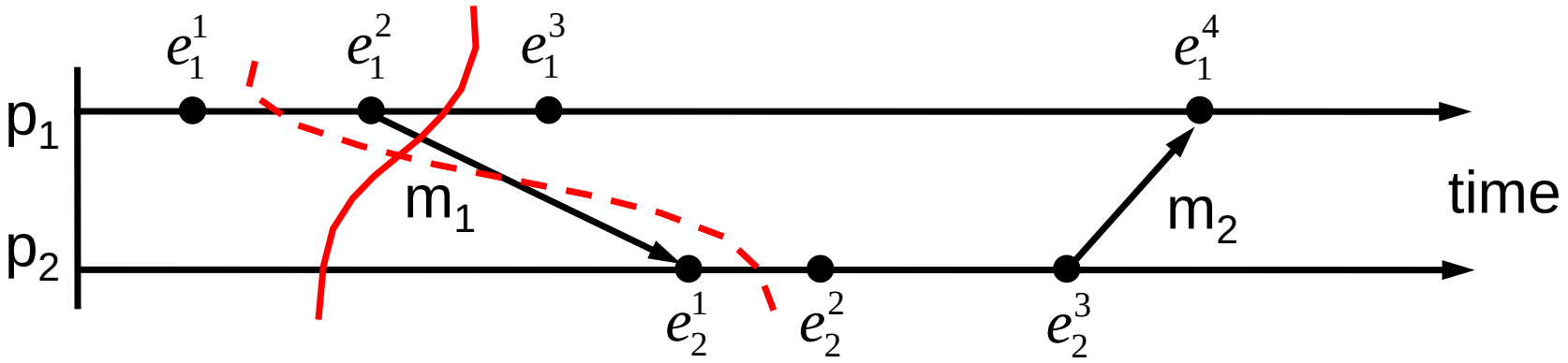


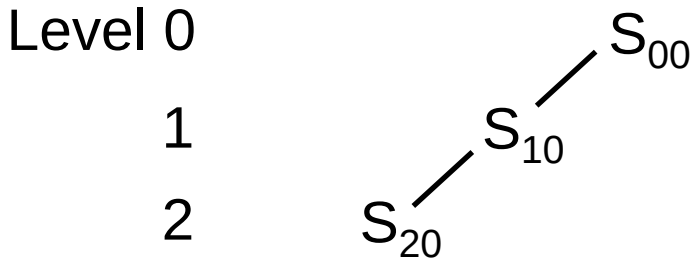
Level 0

1

- Listing all consistent global states through a lattice
  - Global states reachable from S_{00} include S_{10}, S_{01}
  - S_{10} is consistent, S_{01} is not consistent

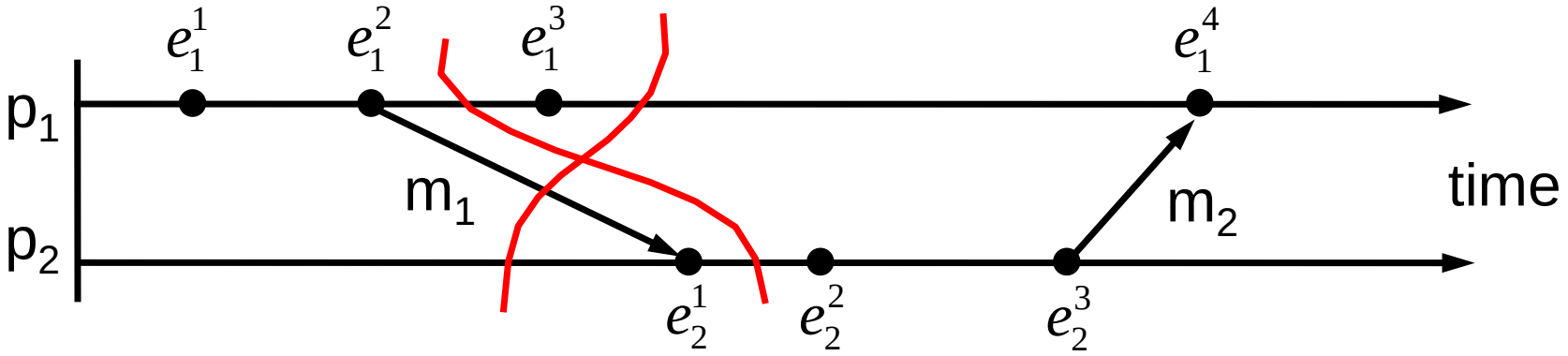
Lattice of Consistent Global States



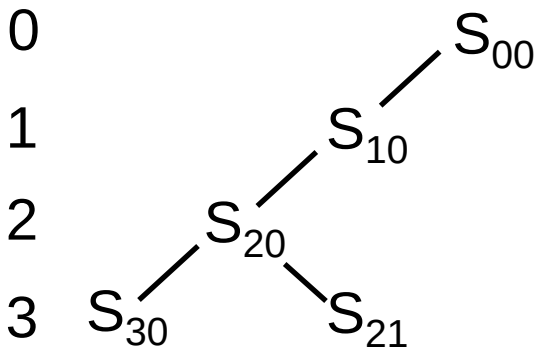


- Listing all consistent global states through a lattice
  - Global states reachable from S_{10} include S_{20}, S_{11}
  - S_{20} is consistent, S_{11} is not consistent

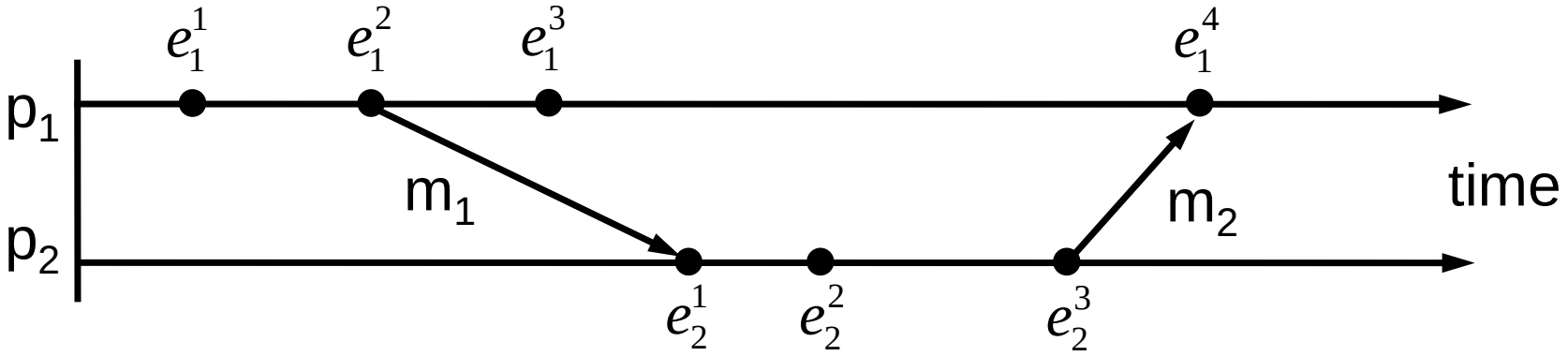
Lattice of Consistent Global States



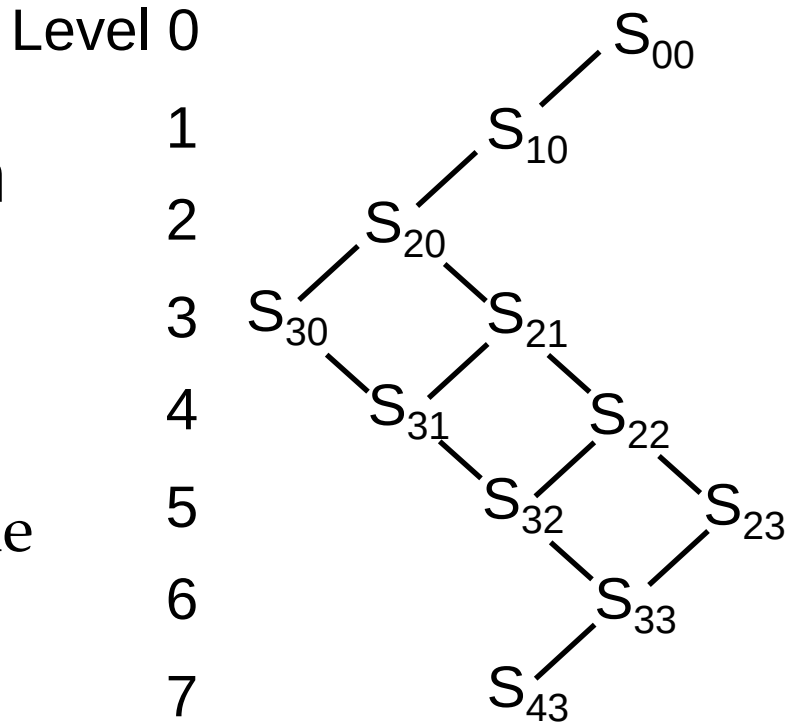
- Listing all consistent global states through a lattice
  - Global states reachable from S_{20} include S_{30}, S_{21}
  - S_{30} and S_{21} are both consistent



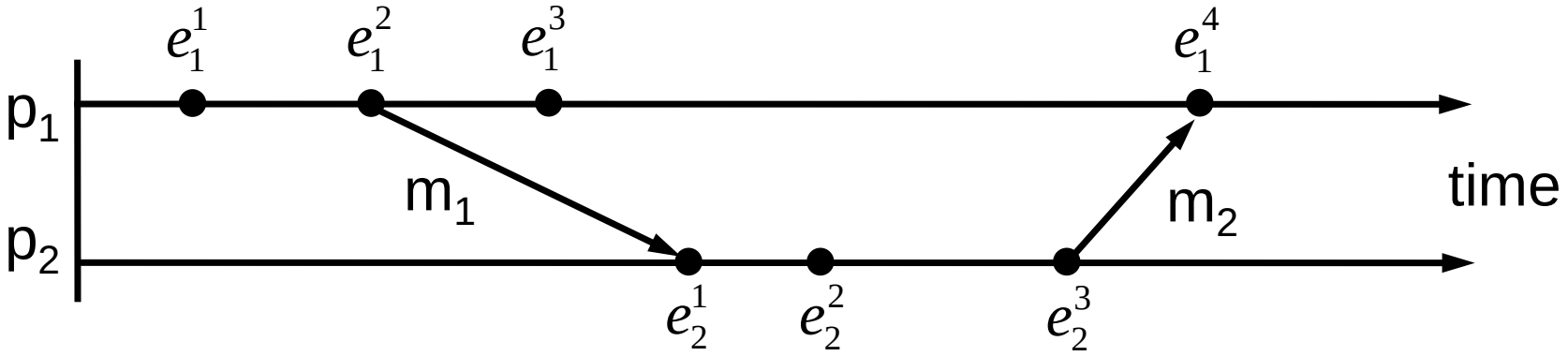
Lattice of Consistent Global States



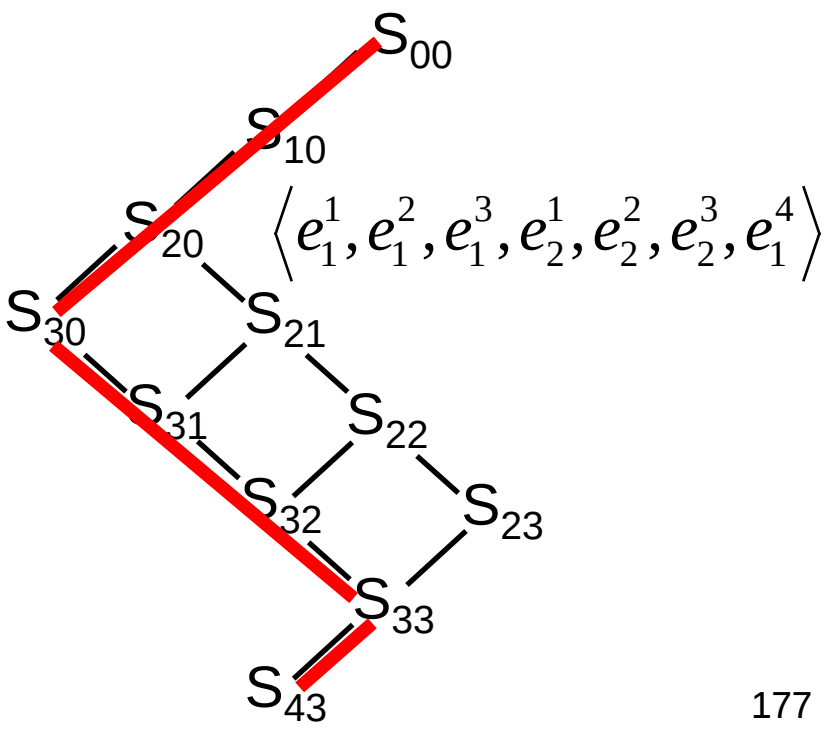
- Listing all consistent global states through a lattice
  - Complete lattice
  - In general, a global state may be reachable from several global states



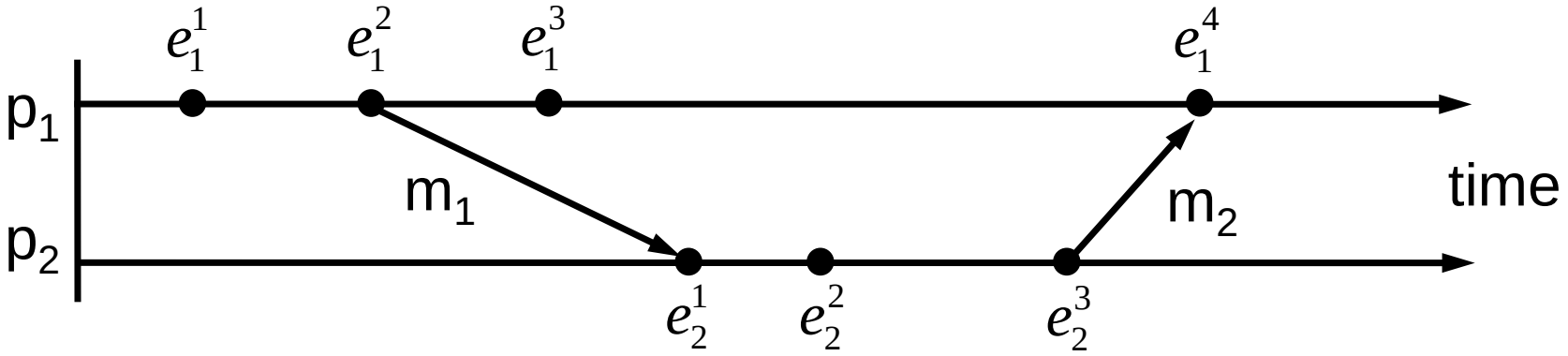
Lattice of Consistent Global States



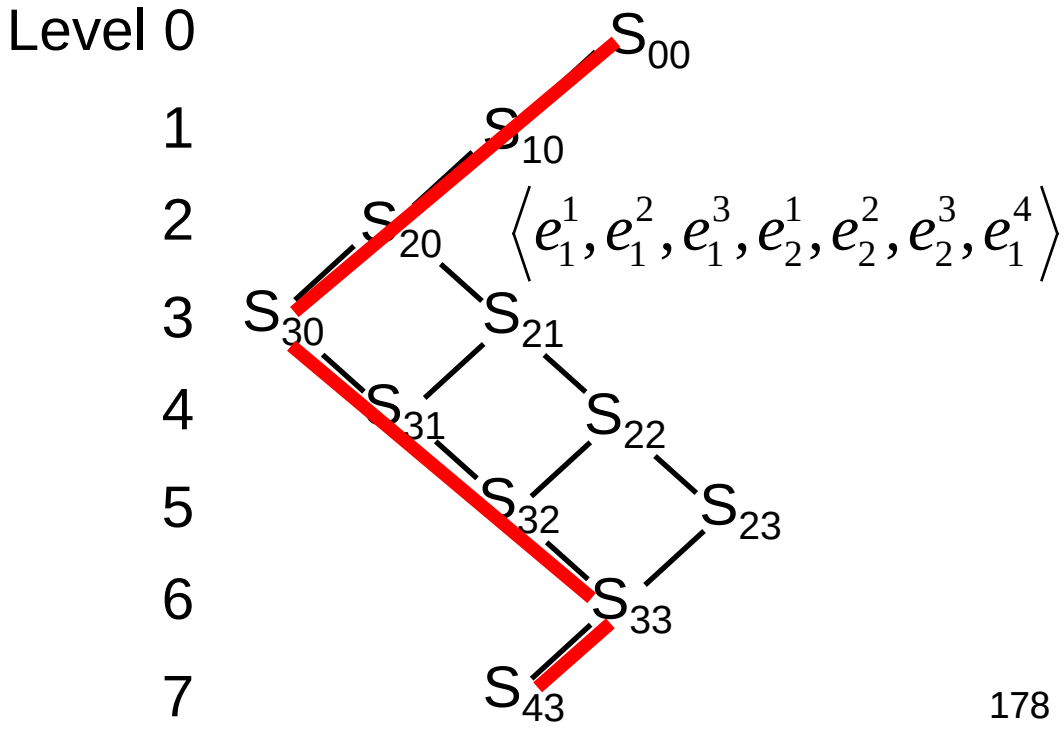
- Each path from the top-level state to the bottom-level state in the lattice represents an ordering of all events that is consistent with causal ordering (i.e., a possible execution of the system)



Lattice of Consistent Global States



- Without global time, we cannot tell which one is the actual execution of the system among all possible executions



Observing Consistent Global States

- Use an additional monitor process that lies outside the system to observe the execution of the system
- All processes in the system send their states to the monitor process
  - Each process p_i sends its initial state to the monitor process initially
  - Then, when an event $e_i^{c_i}$ occurs at p_i, p_i sends a state message (including its new state $s_i^{c_i}$ and the vector clock timestamp of the event) to the monitor process
- The monitor process assembles consistent global states

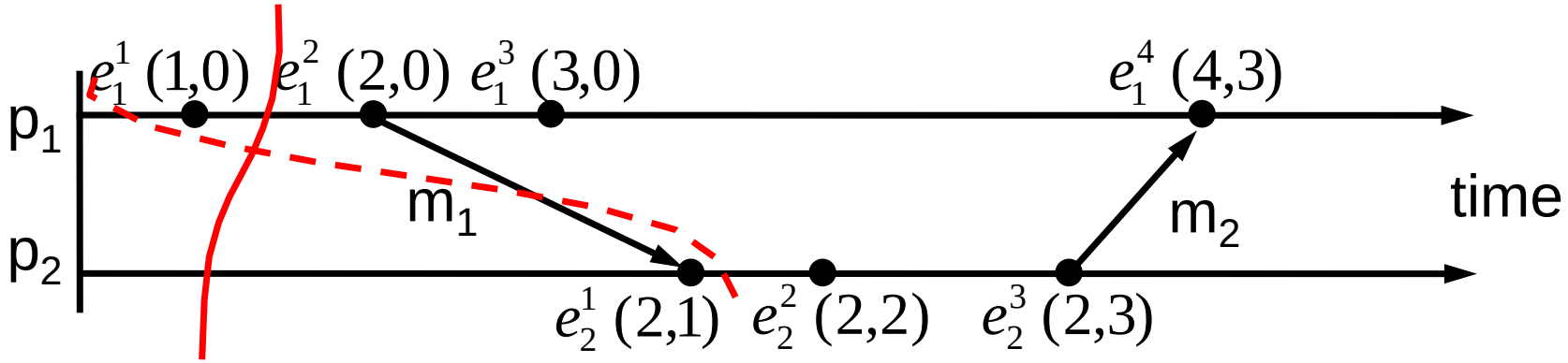
Observing Consistent Global States

- In general, how to assemble consistent global states?
  - $< s_1^{c_1}, s_2^{c_2}, ..., s_N^{c_N} >$: a global state drawn from the state messages
  - $V(e_i^{c_i})$: vector clock timestamp of event $e_i^{c_i}$
  - $< s_1^{c_1}, s_2^{c_2}, ..., s_N^{c_N} >$ is a consistent global state if and only if for any i and j

$$V(e_i^{c_i})[i] \geq V(e_j^{c_j})[i]$$

  - How to prove it?

Example

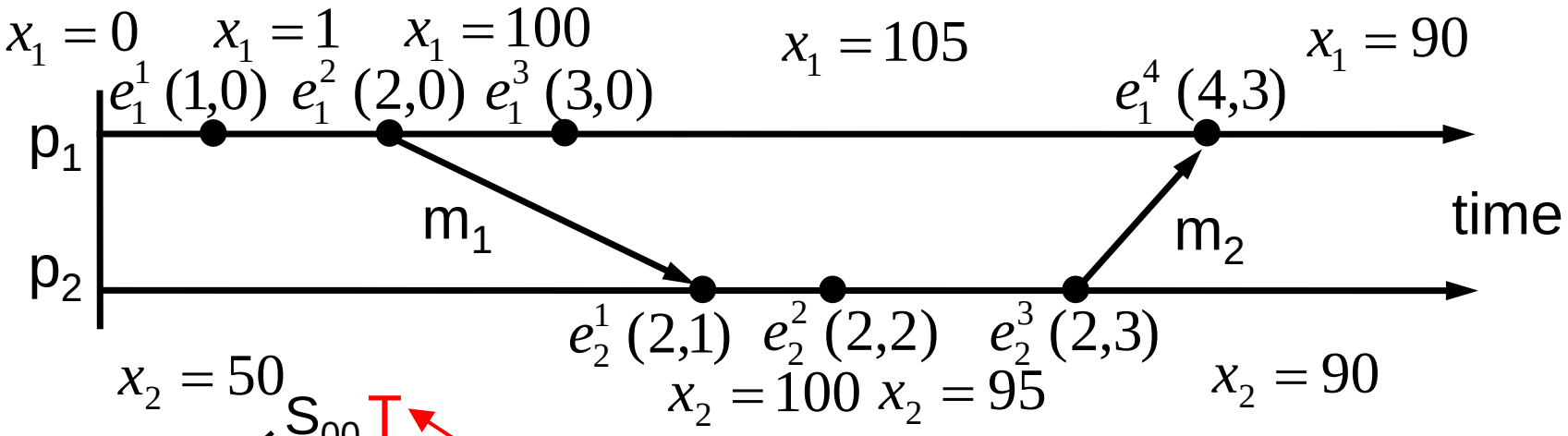


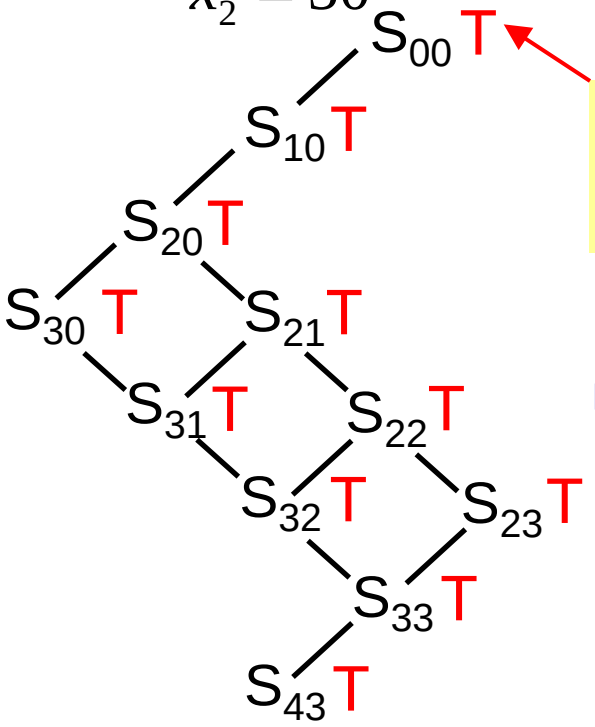
- $S_{ij} = < s_1^i, s_2^j >$: global state after i events at process p_1 and j events at process p_2
- S_{10} is a consistent global state
  - $V(e_1^1) = (1,0)$ and $V(e_2^0) = (0,0)$
- S_{01} is not a consistent global state
  - $V(e_1^0) = (0,0)$ and $V(e_2^1) = (2,1)$

Distributed Debugging

- How to check whether a constraint is broken during execution?
- Construct the lattice of consistent global states
- For each consistent global state, evaluate whether it satisfies the constraint
  - If all states satisfy the constraint, the constraint cannot be broken in the execution
  - If there exists a state not satisfying the constraint, the constraint is possibly broken in the execution
  - If every path from the top-level state to the bottom-level state passes through at least one state not satisfying the constraint, the constraint must be broken in the execution

Distributed Debugging

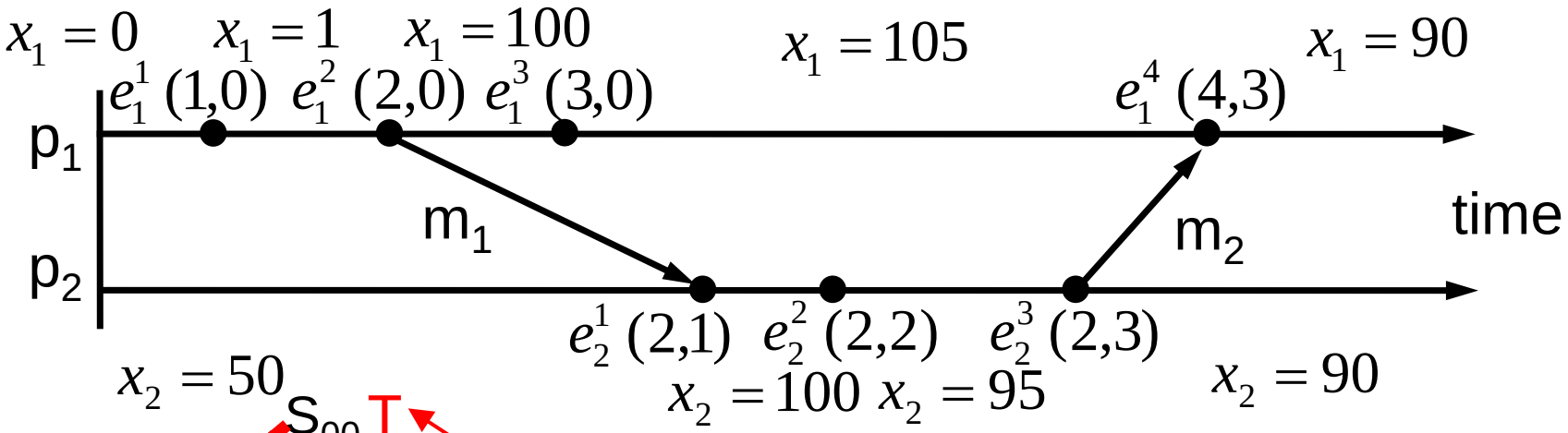


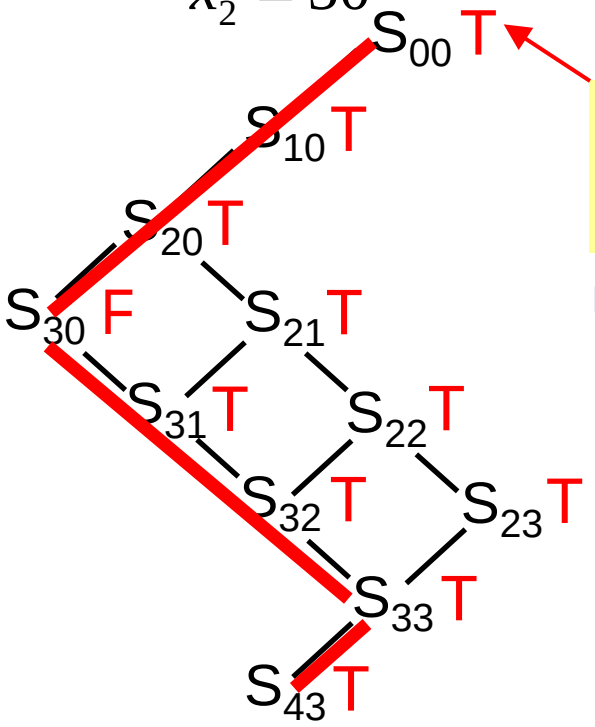


whether the consistent global state satisfies the constraint

- The constraint $|x_1 - x_2| \leq 60$ cannot be broken in the execution
  - All states satisfy the constraint

Distributed Debugging

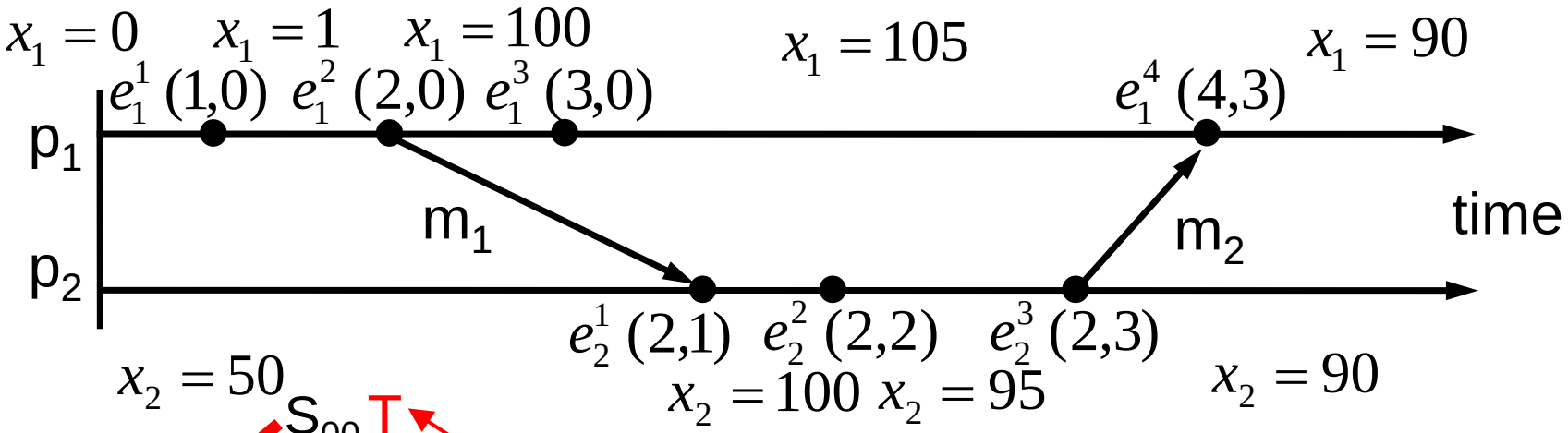


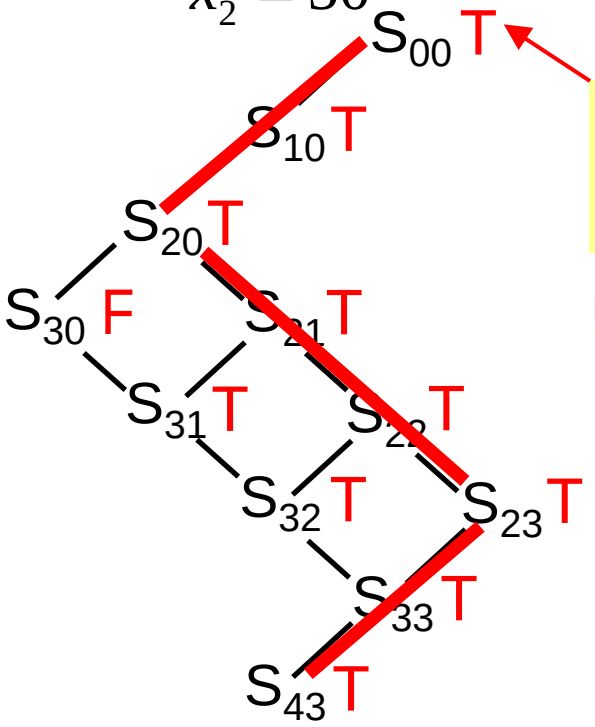


whether the consistent global state satisfies the constraint

- The constraint $|x_1 - x_2| \leq 50$ is possibly broken in the execution
  - There is a state not satisfying the constraint
  - If the actual execution is the path shown left, the constraint is broken

Distributed Debugging

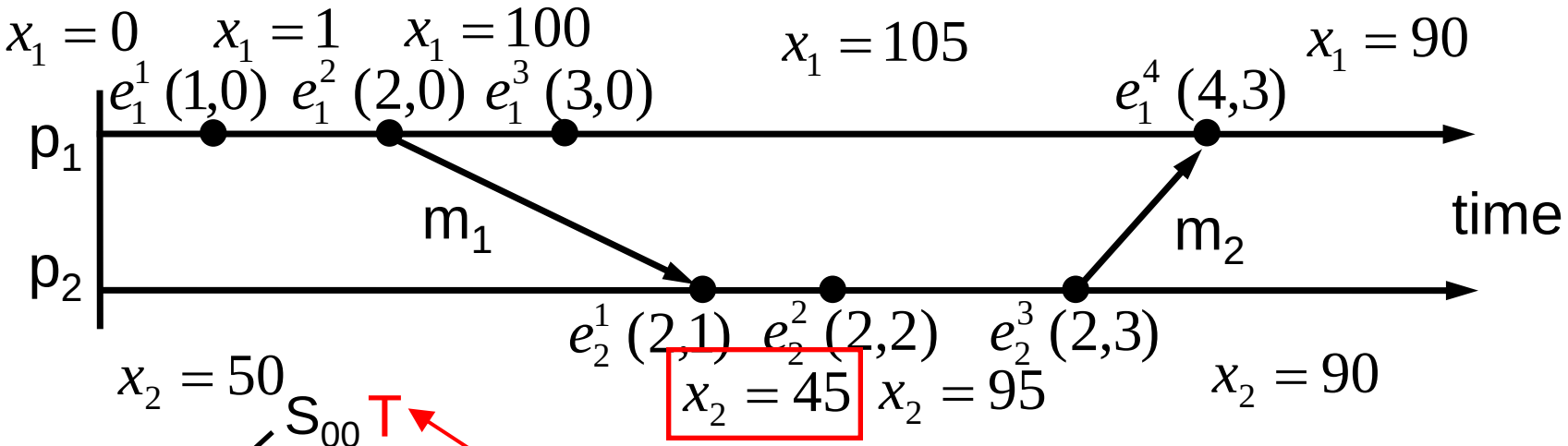


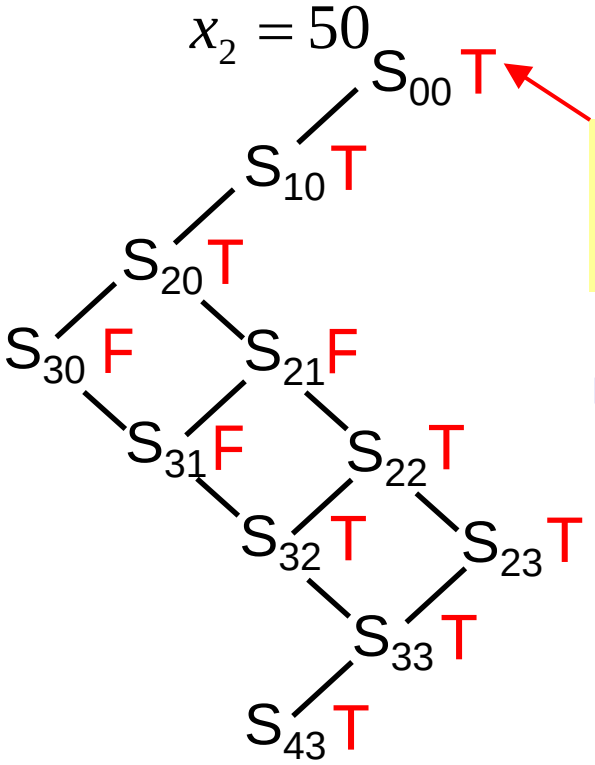


whether the consistent global state satisfies the constraint

- However, we cannot conclude that the constraint $|x_1 - x_2| \leq 50$ must be broken in the execution
  - There is a path from the top-level state to the bottom-level state passing only states satisfying the constraint

Distributed Debugging





whether the consistent global state satisfies the constraint

- The constraint $|x_1 - x_2| \leq 50$ must be broken in the execution
  - Each path from the top-level state to the bottom-level state passes through either S_{30} or S_{21}

Summary

- How to synchronize different computer clocks?
  - Cristian's method
  - Berkeley algorithm
  - Network Time Protocol
  - These methods are based on exchanging clock values and taking into account message transmission times
  - Variations in message transmission times and the way those variations are handled determines the accuracy of synchronization
  - Practically obtainable synchronization accuracy fulfills many requirements but is not sufficient to determine the ordering of arbitrary events at different computers

Summary

- How to order events without global time?
  - Causal ordering
  - Lamport's logical clocks and vector clocks
- How to observe the global state of a distributed system?
  - Cut and consistent cut
  - Chandy and Lamport algorithm to record a snapshot of a distributed system during execution
- Distributed debugging
  - How to enumerate all consistent global states?
  - How to check whether a constraint is broken during execution?